



AMERICAN INTERNATIONAL UNIVERSITY–BANGLADESH (AIUB)

Dept. of Computer Science
Faculty of Science and Technology

CSC2210: OBJECT ORIENTED PROGRAMMING 2

Summer 2023-2024

Section: [AA]

Group No: 00

Project Report On

Project Name [BUS TICKET MANAGEMENT SYSTEM]

Supervised By

Dr. Md. Iftekharul Mobin

Submitted By:

Name				ID	
1. KAZI FAHMID				24-58484-2	
2. JANNATUL MAWA RODELA				24-58743-2	
3.					
Obtained Marks for CO2 and CO3 (Description given in the following page)					
Assessment Criteria	Not Attended/ Incorrect (0)	Inadequate (1-2)	Average (3)	Good (4)	Excellent (5)
Evaluation Criteria (CO2)	Total =		Evaluation Criteria (CO3)		Total =
Requirement fulfillment			Organization of the application		
Validation			Representation and Integration of Database		
Verification			Graphical User Interface		

CO2: Display and verify the mean of a real-life Project using the concepts of C# Graphical User Interface based environment with database integration to depict a desktop-based application.

Assessment Criteria	Not Attended/ Incorrect (0)	Inadequate (1-2)	Average (3)	Good (4)	Excellent (5)
Evaluation Criteria	Evaluation Definition				
Requirement fulfillment	Fails to demonstrate any understanding of real-life scenario-based project development or functional requirement identification. There is no attempt to depict a project or identify functional requirements accurately.	Demonstrates limited understanding of real-life scenario-based project development and functional requirement identification. The project depicted lacks coherence or relevance to real-life scenarios, and functional requirements are inaccurately identified or insufficiently described.	Presents a basic depiction of a real-life scenario-based project and identifies some functional requirements. However, the project lacks depth or complexity, and some functional requirements may be vaguely defined or missing key details.	Effectively demonstrates a realistic scenario-based project and accurately identifies most functional requirements. The project is well-developed with appropriate complexity, and functional requirements are clearly articulated with relevant details.	Exhibits an exceptional understanding of real-life scenario-based project development and accurately identifies all functional requirements. The project is meticulously developed with thorough attention to detail, reflecting a comprehensive understanding of Object-Oriented Programming project development activities.
Validation	Fails to demonstrate any understanding or implementation of validation forms in their system. There is no attempt to deal with data validation, and validation requirements are completely ignored or incorrectly applied.	Demonstrates limited understanding of validation forms and data validation techniques. While some attempt may be made to implement validation, it is incomplete or poorly executed, leading to inadequate handling of data validation.	Shows a basic understanding of validation forms and data validation techniques. They attempt to implement validation, but some aspects may be missing or incorrectly implemented, resulting in partial or inconsistent handling of data validation.	Effectively demonstrates the use of validation forms and implements data validation techniques. Validation is mostly accurate and comprehensive, ensuring the proper handling of data input and verification in the system.	Exhibits an exceptional understanding and implementation of validation forms and data validation techniques. Validation is meticulously implemented with thorough attention to detail, ensuring robust data validation procedures and contributing to the overall reliability and integrity of the system.
Verification	Fails to demonstrate any attempt to verify the system data or functional requirements. There is no evidence of understanding or implementation	Demonstrates limited understanding of verification processes and data flow in the system. Verification attempts are incomplete or	Shows a basic understanding of verification processes and attempts to verify system data. However, verification efforts may be inconsistent or	Identifies and verifies system data, ensuring proper functional requirements are met. Verification efforts are mostly accurate and thorough, with attention to	Exhibits an exceptional understanding of verification processes and meticulously verifies system data. Verification efforts are comprehensive

	of verification processes, and data flow is not considered.	inaccurate, and there is insufficient consideration given to ensuring data integrity and functionality.	lack thoroughness, and there may be gaps in ensuring proper functional requirements and data flow.	ensuring data integrity and appropriate data flow within the system.	and precise, with a keen focus on ensuring all functional requirements are met and maintaining proper data flow throughout the system.
--	---	---	--	--	--

CO3: Prepare and Explain a real life desktop based application synthesizing several component of C# along with development tools to adhere the given requirements.

Assessment Criteria	Not Attended/ Incorrect (0)	Inadequate (1-2)	Average (3)	Good (4)	Excellent (5)
Evaluation Criteria	Evaluation Definition				
Organization of the application	Fails to identify any suitable real time application or requirements for project development activities related to OOP.	Limited understanding about the project scopes and scenarios or identification of functional requirements.	Lacks depth or relevance to OOP project development activities and may contain inaccuracies. Real-life scenarios are mentioned, but the discussion lacks depth or clarity.	Consider and integrate the idea of several core aspects of the project along with relevance to real-life scenarios. Demonstrating a solid understanding of the application presentation.	Generalize and exhibits an exceptional understanding of project preparation according to a to real-life scenarios. Also contains proper and insightful identification of the system which is comprehensive and precise.
Representation and Integration of Database	Fails to identify and present any understanding or implementation of database. Also failed to integrate the data with the project itself.	Limited understanding of the database concepts or their proper way of using in a real time project. While some attempt may be made to implement but it is incomplete or poorly executed, leading to inadequate design.	Lacks depth or relevance to database integration with the application. Shows a basic understanding but some aspects may be missing or incorrectly implemented, resulting in partial or inconsistency. May lack proper normalization.	Integrate the database with the forms properly and implements it with proper validation which is mostly accurate and comprehensive, ensuring the proper handling of data input and verification along with general normalization.	Exhibits an exceptional understanding and implementation of database ensuring attention to detail, and robust data manipulation procedures and contributing to the overall clarity.
Graphical User Interface	Fails to present or prepare GUI based application interfaces. There is no evidence of creating or integrating such things according to their usefulness.	Limited understanding of graphical user interfaces. Lack of design knowledge. Very poor attempt to make such things which are currently obsolete or can't be identified as coherent.	Shows a basic understanding of creating user interfaces. Most of them are interconnected but maybe some of them lack it. However, most of it can be described as user friendly.	Effectively identifies and meet the consider the simplicity. Design related works are mostly accurate and taken proper attention to ensuring a user-friendly coherent system.	Exhibits an exceptional work design following a high standard of simple and elegant work. Several controls and mechanism has been organized in a preferred way according to the coherent usage .

CSC 2210 — Object Oriented Programming 2

Project Assignment 01: System Design & Report Submission

 Bus Ticket Management System — Desktop Application

Chapter 1: Introduction and Case Study:

In the world of bus services, BusGo helps bus operators to sell tickets to passengers via a centralized desktop application. Suppose Rahim wants to go to

Chattogram from Dhaka; instead of going to various bus counters and finding which bus counter has a ticket, Rahim can search from the Bus Ticket Management System for BusGo. He will see all available trips by searching according to the destination, date of journey, fare, and bus operators. He will choose the most appropriate one and check its description and reviews. He can select his desired seats and book the tickets. He can make payments online or pay the amount on booking. After confirmation, the system will issue his ticket in printed form along with an invoice.

On the other hand, Karim Travels is a bus operator in BusGo. He operates his own desktop and uses the Bus Ticket Management System as his centralized system. He registers his buses, chooses the seat plan, selects routes and schedules, and sets the fares. When Rahim booked the ticket, it automatically updated the availability of seats in the system. Karim will be able to see who has booked his bus, track ticket sales and revenue, and manage special offers and discounts. Karim is only able to view his bus, trips, bookings, sales, and revenue in the system. He cannot view any information about other bus operators' data.

The Super Admin takes care of everything in the Bus Ticket Management System for BusGo. He ensures that everything runs smoothly in the application. Whenever a new bus operator requests to join, the Super Admin approves or denies the request. Super Admin can view customers and operators, revenue and commission, low-rated operators, categories, and reviews and complaints. All customers can review and rate their experience on the Bus ticket management system, which helps future customers when making decisions and helps operators improve their performance. Additionally, the system handles all cancellations and refunds for tickets based on the given time frame.

When Rahim cancels his ticket, the system will refund him depending on a particular day before embarkation. If Rahim cancels his ticket one day before, the system will refund him 90% of the amount he paid. But, if he cancels it 12 hours to 24 hours before, he will get a 50% refund. However, if he cancels within 12 hours of embarkation, he will not get a refund. The system will record all the details of the booking, including the customer's payment, cancellations, and refunds.

Moreover, the payment process works accordingly when customers buy tickets. The system's wallet gets the ticket money, including its commission, for all successful transactions. On the other hand, the bus operator also gets his share of the ticket money.

Behind these interactions, the system is built around a small set of core entities. Every bus operator is uniquely identified by an OperatorID and holds details such as company name, contact information, approval status, and commission rate. Each operator can register multiple buses, and every bus is identified by a BusID and stores its registration number, seat capacity, category, and seat layout. A bus is scheduled onto one or more trips, where each trip has a TripID, a route, a travel date, departure and arrival times, and a fare. Every route itself is defined by a

departure location and a destination, and one route can be reused across many trips. When an operator creates a trip, the system automatically generates individual **TripSeat** records for every seat based on the bus capacity. When a customer books seats, the system creates a **Booking** record (holding payment status and total amount) and links the selected **TripSeat** entries to that specific **BookingID**, and one customer can make many bookings over time. Each customer is identified by a CustomerID with a name, contact details, and login credentials. After a completed trip, a customer may leave a review and rating, and these are linked back to both the customer who wrote them and the operator or trip they describe — meaning one operator can accumulate many reviews across many trips. Finally, every successful booking generates a payment record that ties an operator's earnings and the platform's commission back to that single transaction.

Chapter 2 — Functional Requirements & User Stories

2.1 Introduction

The Bus Ticket Management System is a desktop-based application developed to manage bus operators, buses, routes, schedules, bookings, seats, payments, and ticket-related information. The system provides different functionalities according to the responsibilities of three user roles: Super Admin, Operator/Admin, and Customer/Ticket Buyer.

Each role has a separate area of the system with appropriate access to ensure that users can only perform activities relevant to their responsibilities.

2.2 User Roles

The system consists of three primary user roles.

2.2.1 Super Admin

The Super Admin represents the system owner or central administrator. This role has the highest level of access and is responsible for monitoring and controlling the overall system.

The Super Admin can:

- Log in securely to the system.
- Manage bus operators.
- View overall system statistics.
- Monitor registered buses.
- Monitor ticket sales.

- Monitor platform commission/revenue.
- View operator commission payment status.
- Identify received, pending, and overdue operator payments.
- Generate and view system reports.
- Monitor the overall performance of the bus ticket platform.

2.2.2 Operator / Admin

The Operator/Admin represents a bus company or bus-service operator using the platform to manage its transportation services.

The Operator/Admin can:

- Log in securely.
- Manage their buses.
- Add, update, and remove bus information.
- Manage routes.
- Create and update travel schedules.
- View customer bookings related to their services.
- View payment and report information.
- Manage bus-service-related information.
- Monitor the schedules and bookings associated with their buses.

2.2.3 Customer / Ticket Buyer

The Customer/Ticket Buyer is the end user who uses the system to search for and purchase bus tickets.

The Customer can:

- Register a new account.
- Log in to the system.
- Search for available buses.
- View available schedules.
- Select available seats.

- Book tickets.
- Make payments for bookings.
- View booking information.
- View payment information and booking status.

2.3 User Stories

User stories describe the system requirements from the perspective of the users. Each story follows the format:

As a [user], I want to [action] so that [benefit].

2.3.1 Super Admin User Stories

US-SA-01 — Super Admin Login

As a Super Admin, I want to log in using my credentials so that only authorized administrators can access system management features.

US-SA-02 — Manage Operators

As a Super Admin, I want to manage bus operators so that I can control which operators are allowed to use the system.

US-SA-03 — View System Statistics

As a Super Admin, I want to view system statistics so that I can understand the overall activity and performance of the platform.

US-SA-04 — Monitor Buses

As a Super Admin, I want to monitor registered buses so that I can keep track of the transportation services available through the system.

US-SA-05 — Monitor Ticket Sales

As a Super Admin, I want to monitor ticket sales so that I can evaluate the overall sales activity of the platform.

US-SA-06 — Monitor Platform Commission

As a Super Admin, I want to view platform commission information so that I can monitor the revenue generated by the system.

US-SA-07 — Monitor Operator Payments

As a Super Admin, I want to view operator payment status so that I can identify which operators have paid, have pending payments, or have overdue commission payments.

US-SA-08 — View Reports

As a Super Admin, I want to generate and view reports so that I can analyze ticket sales, commission, and operator payment information.

2.3.2 Operator / Admin User Stories

US-OA-01	—	Operator	Login
As an Operator/Admin, I want to log in using my credentials so that I can securely access my bus-service management features.			
US-OA-02	—	Manage	Buses
As an Operator/Admin, I want to add, update, and manage my buses so that customers can see accurate bus-service information.			
US-OA-03	—	Manage	Routes
As an Operator/Admin, I want to manage routes so that I can define the destinations and travel paths offered by my bus service.			
US-OA-04	—	Manage	Schedules
As an Operator/Admin, I want to create and manage bus schedules so that customers can view available travel dates and times.			
US-OA-05	—	View	Bookings
As an Operator/Admin, I want to view customer bookings so that I can monitor tickets booked for my bus services.			
US-OA-06	—	View	Payment Information
As an Operator/Admin, I want to view payment and report information so that I can monitor financial activity related to my services.			
US-OA-07	—	Manage	Bus-Service Information
As an Operator/Admin, I want to maintain my bus-service information so that customers receive accurate information about available services.			

2.3.3 Customer / Ticket Buyer User Stories

US-CB-01	—	Customer	Registration
As a customer, I want to register an account so that I can use the ticket booking services provided by the system.			
US-CB-02	—	Customer	Login
As a customer, I want to log in using my credentials so that I can securely access my booking and payment features.			
US-CB-03	—	Search	Available Buses
As a customer, I want to search available buses by route and travel date so that I can select a suitable bus.			
US-CB-04	—	View	Schedules
As a customer, I want to view departure and arrival information so that I can choose a convenient travel schedule.			

US-CB-05	—	Select	Seats
As a customer, I want to view and select available seats so that I can choose my preferred seats before booking.			
US-CB-06	—	Book	Tickets
As a customer, I want to book selected seats so that I can reserve a place on my chosen bus.			
US-CB-07	—	Make	Payment
As a customer, I want to make payment for my booking so that my ticket reservation can be confirmed.			
US-CB-08	—	View	Booking Information
As a customer, I want to view my booking information so that I can check my reserved tickets and travel details.			
US-CB-09	—	View	Payment Information
As a customer, I want to view my payment information so that I can check the payment status associated with my booking.			

2.4 Functional Requirements

Functional requirements describe what the system must be able to do.

FR-01 — User Authentication

The system shall provide secure login functionality for Super Admin, Operator/Admin, and Customer users.

FR-02 — Customer Registration

The system shall allow new customers to create an account by providing the required registration information.

FR-03 — Role-Based Access

The system shall identify the user's role and provide access only to the functionalities permitted for that role.

FR-04 — Operator Management

The system shall allow the Super Admin to view, add, update, and manage operator information.

FR-05 — Bus Management

The system shall allow Operators/Admins to manage their bus information.

FR-06 — Route Management

The system shall allow Operators/Admins to create and manage bus routes.

FR-07 — Schedule Management

The system shall allow Operators/Admins to create and manage schedules containing travel dates, departure times, arrival times, buses, routes, fares, and schedule status.

FR-08 — Bus Search

The system shall allow customers to search for available buses based on travel-related criteria such as route and travel date.

FR-09 — Schedule Viewing

The system shall display available bus schedules to customers, including relevant travel and bus information.

FR-10 — Seat Selection

The system shall allow customers to view available seats and select seats before completing a booking.

FR-11 — Ticket Booking

The system shall allow customers to create a booking by selecting an available schedule and seat(s).

FR-12 — Payment Processing

The system shall allow customers to complete payment for their selected booking and update the corresponding booking/payment status.

FR-13 — Booking Management

The system shall store booking information and allow authorized users to view relevant booking records.

FR-14 — Payment Information

The system shall maintain payment information associated with customer bookings.

FR-15 — Commission Management

The system shall calculate and maintain the platform commission associated with ticket sales.

FR-16 — Operator Payment Monitoring

The system shall allow the Super Admin to monitor operator commission payments and identify their payment status.

FR-17 — System Statistics

The system shall provide the Super Admin with statistics such as total operators, buses, tickets sold, sales, and platform commission/revenue.

FR-18 — Reporting

The system shall provide report information related to ticket sales, commission, and operator payments.

FR-19 — Data Validation

The system shall validate required user input before storing or processing information.

FR-20 — Database Storage

The system shall store user, customer, operator, bus, route, schedule, seat, booking, booking-seat, payment, operator-payment, review, and notification information in the database.

2.5 Non-Functional Requirements

Non-functional requirements define the quality, performance, security, and usability characteristics expected from the system.

NFR-01 — Security

The system should restrict access to role-specific features and prevent unauthorized users from accessing administrative or operator functions.

NFR-02 — Usability

The system should provide a simple and understandable graphical user interface so that users can perform their tasks without unnecessary complexity.

NFR-03 — Reliability

The system should maintain accurate booking and payment information and should minimize data inconsistencies during transactions.

NFR-04 — Data Integrity

The system should maintain relationships between users, customers, operators, buses, routes, schedules, bookings, seats, and payments using appropriate database constraints.

NFR-05 — Performance

The system should respond to common operations such as login, bus searching, schedule viewing, booking, and data retrieval within a reasonable amount of time.

NFR-06 — Maintainability

The system should be organized into separate forms, classes, database tables, and functional modules so that future modifications can be implemented efficiently.

NFR-07 — Availability

The application should be available to authorized users whenever the application and its associated database are running.

NFR-08 — Scalability

The system should be designed so that additional operators, buses, routes, schedules, customers, bookings, and payment records can be added without requiring major changes to the existing system structure.

NFR-09 — Accuracy

The system should accurately calculate booking totals, ticket quantities, sales information, and platform commission.

NFR-10 — Transaction Consistency

Booking and payment operations should preserve database consistency so that a booking is not incorrectly confirmed or duplicated when the transaction fails.

2.6 Business Rules and Constraints

The following business rules define important restrictions and relationships within the system.

1. Each user must have an assigned role such as Customer, Operator, or Super Admin.
2. Only authorized Super Admin users can access system-level administrative functions.
3. Operators/Admins can manage the bus-service information associated with their operations.
4. Customers can only select seats that are available for the selected schedule.
5. A seat selected for a confirmed booking should not be available for another customer for the same schedule.
6. A booking must be associated with a valid customer and schedule.
7. Payment information must be associated with the appropriate booking.
8. Platform commission is calculated from ticket sales according to the system's defined commission rate.

9. Operator payment records are used to monitor whether the commission amount has been received or remains pending/overdue.
10. Required information must be validated before records are inserted or updated in the database.
11. Database relationships must be maintained using primary keys and foreign keys to preserve data integrity.
12. Different bus services may have different seat arrangements, therefore seat availability is managed through the system's seat information rather than assuming a single fixed bus layout.

2.7 Summary

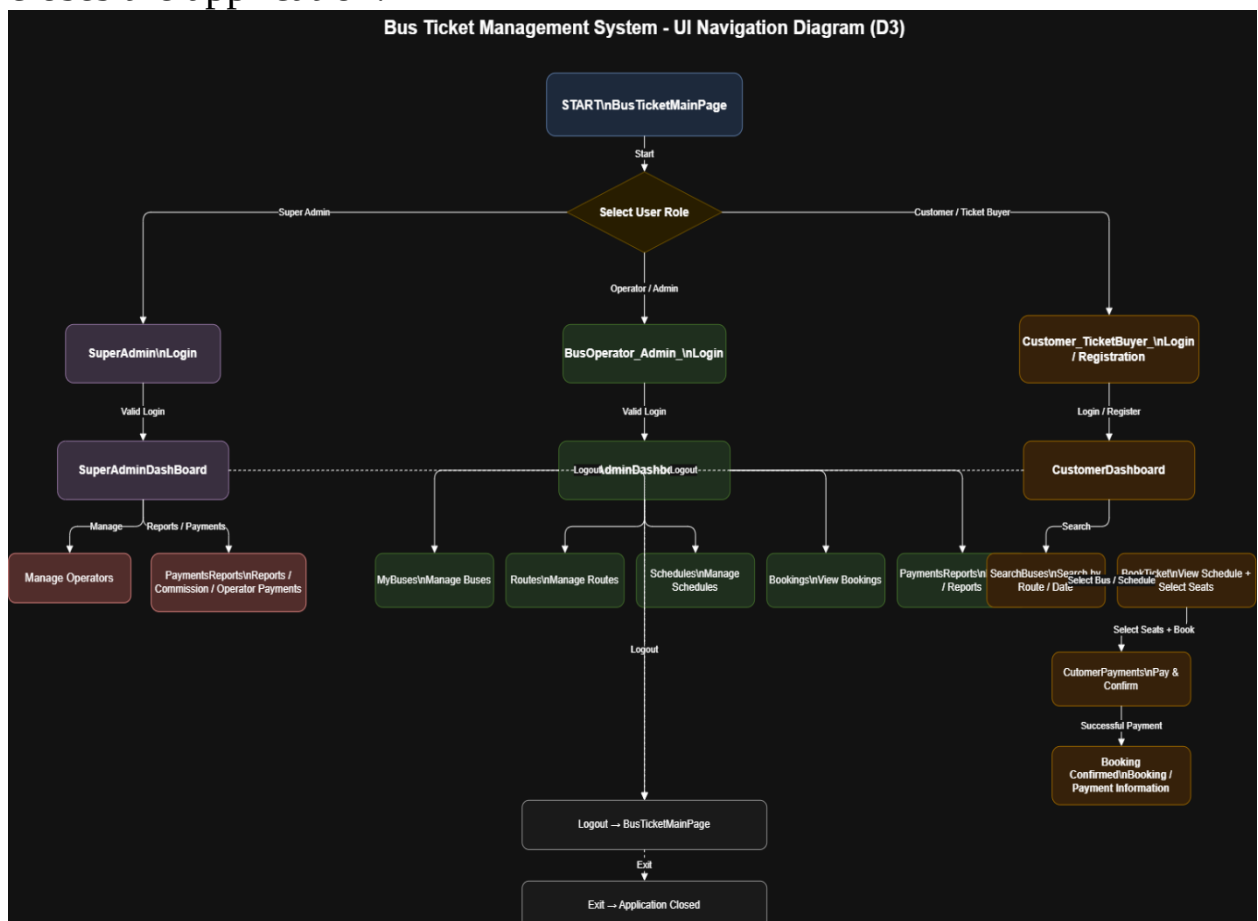
The functional requirements of the Bus Ticket Management System are divided according to the responsibilities of the three main roles. The Super Admin manages and monitors the overall platform, operators manage their buses, routes, schedules, bookings, and related financial information, while customers search for buses, select seats, book tickets, make payments, and view their booking information.

The user stories provide a user-centered representation of these requirements, while the functional requirements define the specific operations that the system must perform. The non-functional requirements ensure that the system remains secure, reliable, usable, maintainable, and accurate. Together, these requirements establish a clear foundation for the design and implementation of the Bus Ticket Management System.

Chapter 3: UI Navigation Flow Explanation:

The application starts from the BusTicketMainPage, where the user selects one of the three available roles: Super Admin, Operator/Admin, or Customer/Ticket Buyer. Based on the selected role, the system navigates to the corresponding authentication interface. A Super Admin is directed to the SuperAdmin login page and, after successful authentication, enters the

SuperAdminDashboard, where system statistics, operators, buses, ticket sales, platform commission, operator payments, and reports can be monitored. An Operator/Admin logs in through BusOperator_Admin_ and is redirected to AdminDashboard, from which they can manage buses, routes, schedules, bookings, and payment/report information. A Customer can either register or log in through the customer authentication interface and then enters the CustomerDashboard. From there, the customer can search for buses and schedules, select available seats through BookTicket, create a booking, and proceed to CustomerPayments to complete the payment and confirm the booking. The customer can also view relevant booking and payment information. At any stage, users can use navigation controls such as cancel or logout to return to the appropriate previous page or the main page, while the exit option closes the application.



Chapter 4 — Database Design

4.1 Database Overview

The Bus Ticket Management System uses a relational database named **BusTicketManagementSystem**. The database is implemented using Microsoft SQL Server and is responsible for storing and managing information related to users, customers, operators, buses, routes, schedules, seats, bookings, payments, reviews, notifications, and operator commission payments.

The database follows a relational structure in which individual entities are represented by separate tables and relationships between entities are established using primary keys and foreign keys.

The major entities in the database are:

- Users
- Customers
- Operators
- Buses
- Routes
- Schedules
- Seats
- Bookings
- BookingSeats
- Payments
- OperatorPayments
- Reviews
- Notifications

The database also uses several integrity constraints, including:

- Primary keys
- Foreign keys
- Unique constraints
- Check constraints
- Default values
- Identity columns

These constraints help maintain data accuracy, consistency, and referential integrity throughout the application.

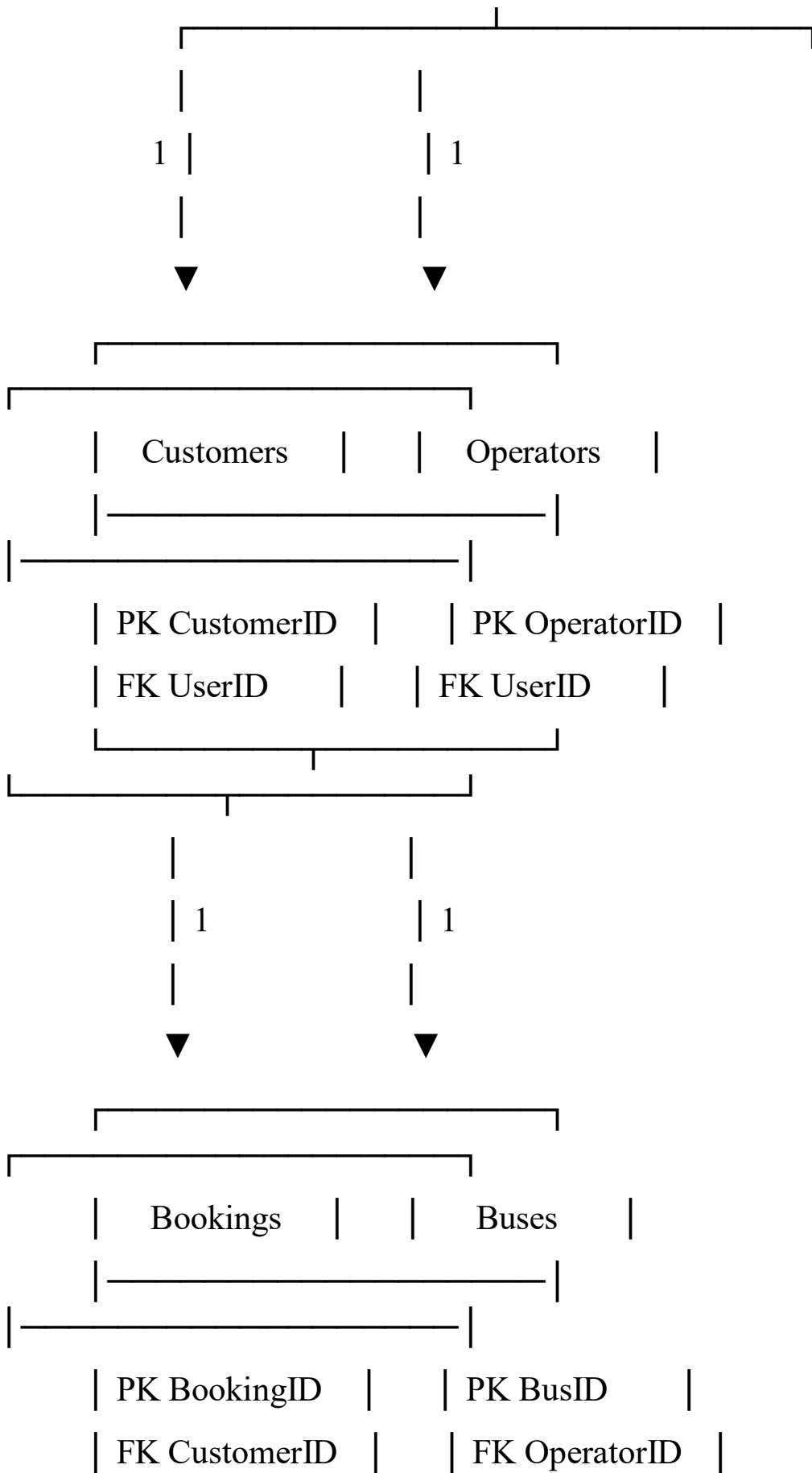
4.2 Database Schema Diagram

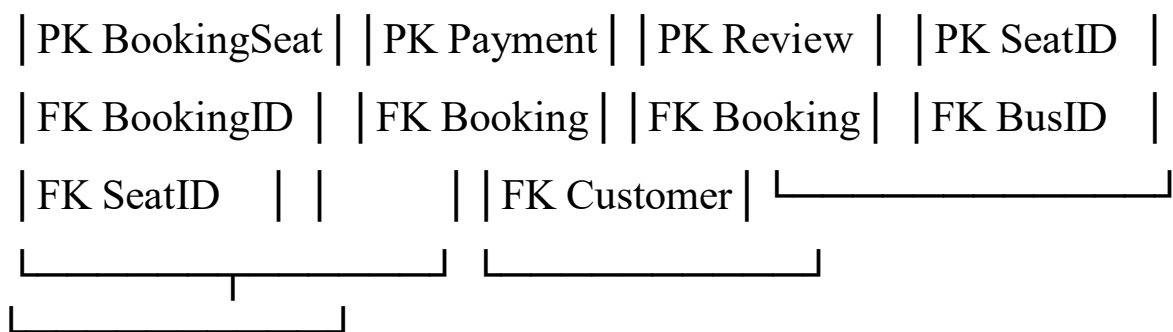
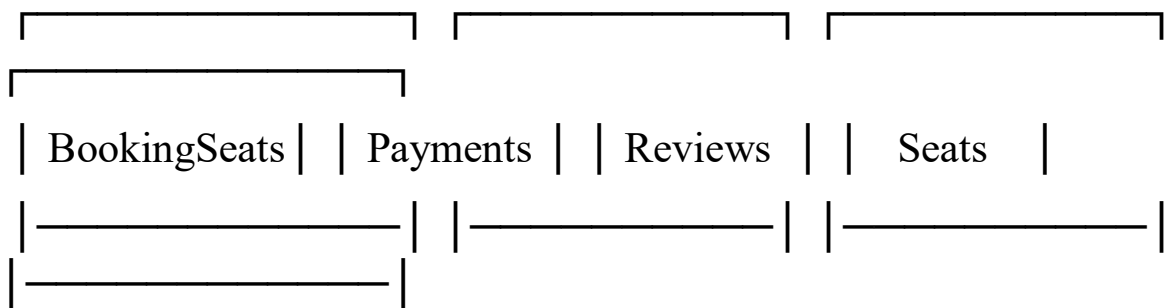
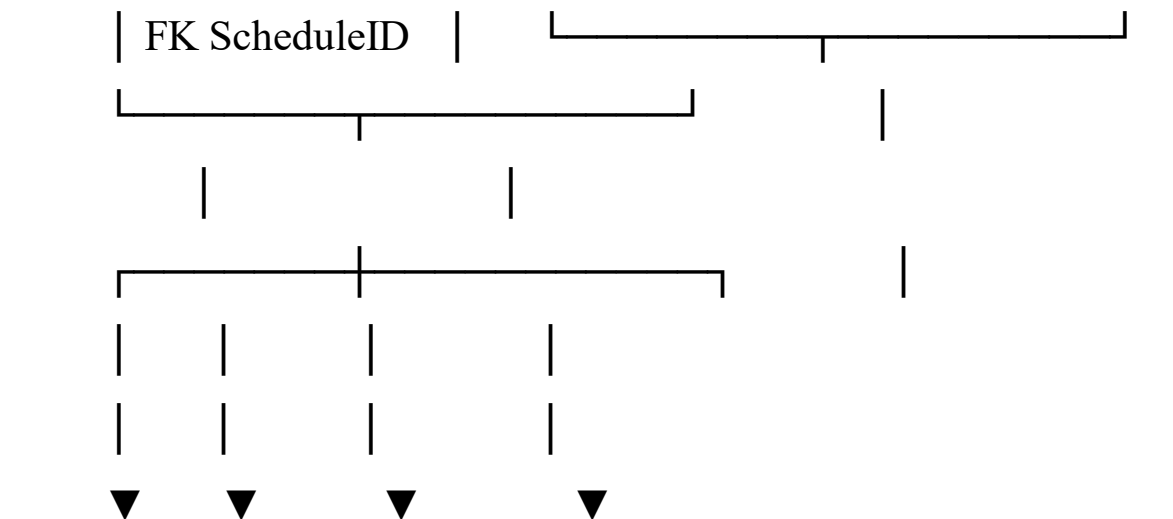
The database contains relationships between the main entities of the Bus Ticket Management System.

The central user table is **Users**. Customer and Operator profiles are connected to the Users table through UserID. A user can therefore have a corresponding customer or operator profile.

The major relationship structure can be summarized as follows:

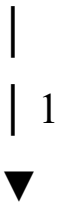
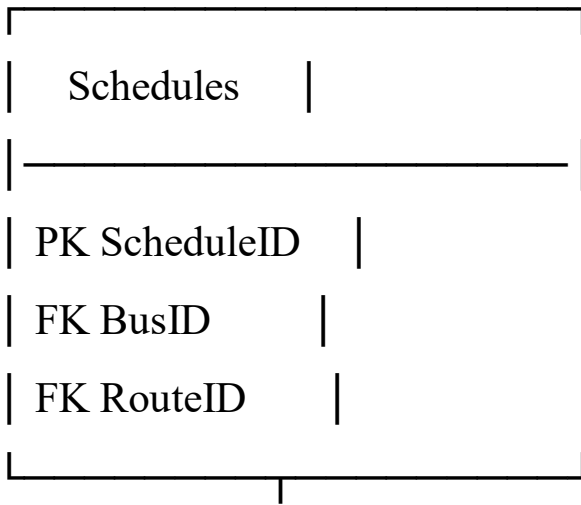
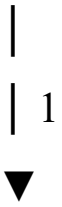
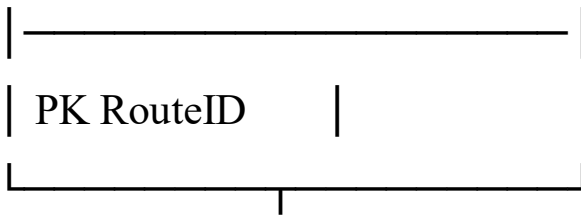
Users	
PK UserID	
Username	
Role	
Status	





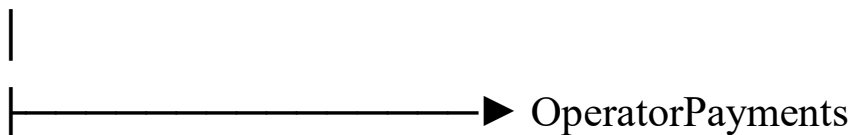
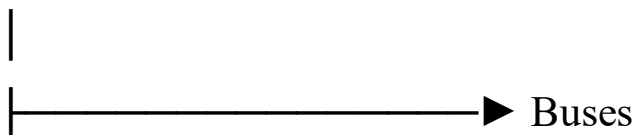
Seats

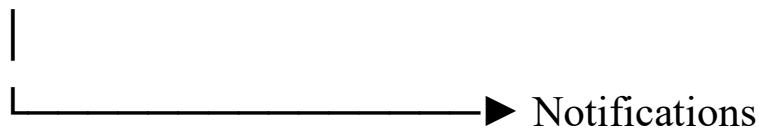




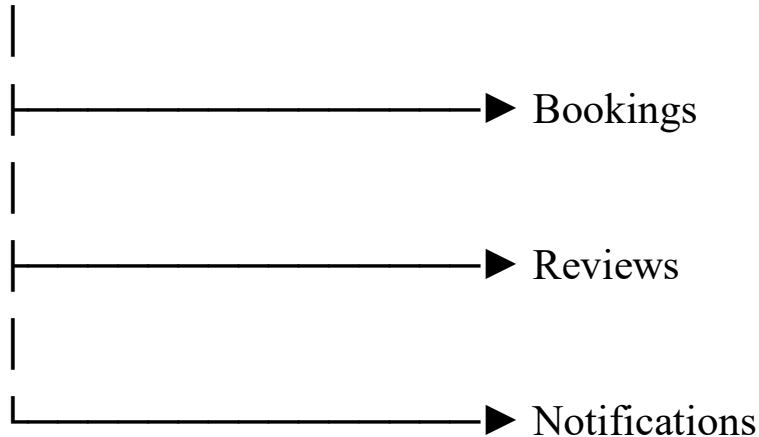
Bookings

Operators





Customers



Main Cardinalities

Relationship	Cardinality Explanation	
Users → Customers	1 : 0..1	A user may have one customer profile
Users → Operators	1 : 0..1	A user may have one operator profile
Operators → Buses	1 : Many	One operator can manage multiple buses
Buses → Seats	1 : Many	One bus contains multiple seats
Buses → Schedules	1 : Many	A bus can operate multiple schedules
Routes → Schedules	1 : Many	A route can be used by multiple schedules

Relationship	Cardinality	Explanation
Customers → Bookings	1 : Many	A customer can make multiple bookings
Schedules → Bookings	1 : Many	A schedule can have multiple bookings
Bookings → BookingSeats	1 : Many	One booking can contain multiple selected seats
Seats → BookingSeats	1 : Many	A seat can appear in booking records
Bookings → Payments	1 : Many	A booking can have payment records
Bookings → Reviews	1 : 0..1	A booking can have at most one review
Customers → Reviews	1 : Many	A customer can submit multiple reviews
Operators → OperatorPayments	1 : Many	An operator can have multiple payment records
Customers → Notifications	1 : Many	A customer can receive multiple notifications
Operators → Notifications	1 : Many	An operator can receive multiple notifications

4.3 Table-by-Table Documentation

4.3.1 Users Table

The Users table is the main authentication and user-account table. It stores login credentials, personal information, role information, and account status.

Column	Data Type	Constraint	Description
UserID	INT	Primary Key, Identity	Unique identifier for each user
Username	NVARCHAR(50)	Unique, NOT NULL	User's login username
Password	NVARCHAR(255)	NOT NULL	User authentication password
FullName	NVARCHAR(100)	NOT NULL	Full name of the user
Email	NVARCHAR(100)	NULL	User email address
Phone	NVARCHAR(20)	NULL	User phone number
Role	NVARCHAR(20)	NOT NULL, CHECK	Defines Customer, Operator, or SuperAdmin
Status	NVARCHAR(20)	NOT NULL, CHECK, DEFAULT	Defines Active or Inactive account
CreatedAt	DATETIME	NOT NULL, DEFAULT	Date and time the account was created

Important Constraints

Username is unique to prevent duplicate login accounts.

The Role column accepts only:

- Customer

- Operator
- SuperAdmin

The Status column accepts:

- Active
- Inactive

4.3.2 Customers Table

The Customers table stores customer-specific information separately from general user-account information.

Column	Data Type	Constraint	Description
CustomerID	INT	Primary Key, Identity	Unique customer identifier
UserID	INT	Foreign Key, Unique, NOT NULL	References Users.UserID
Address	NVARCHAR(255)	NULL	Customer address
CreatedAt	DATETIME	NOT NULL, DEFAULT	Customer profile creation date

Relationship

Customers.UserID references Users.UserID.

The unique constraint on UserID ensures that one user account cannot have multiple customer profiles.

4.3.3 Operators Table

The Operators table stores information about bus-service operators.

Column	Data Type	Constraint	Description
OperatorID	INT	Primary Key, Identity	Unique operator identifier
UserID	INT	Foreign Key, Unique, NOT NULL	References Users.UserID
CompanyName	NVARCHAR(100)	NOT NULL	Operator/company name
CompanyAddress	NVARCHAR(255)	NULL	Company address
LicenseNumber	NVARCHAR(50)	NULL	Operator license number
CommissionRate	DECIMAL(5,2)	NOT NULL, CHECK, DEFAULT	Platform commission percentage
Status	NVARCHAR(20)	NOT NULL, CHECK, DEFAULT	Operator account status
CreatedAt	DATETIME	NOT NULL, DEFAULT	Operator profile creation date
Phone	VARCHAR(30)	NULL	Company phone number

Column	Data Type	Constraint	Description
Email	VARCHAR(100)	NULL	Company email address

Important Constraints

CommissionRate must be between 0 and 100.

Status can be:

- Active
- Inactive

UserID is unique, ensuring that one user account has at most one operator profile.

4.3.4 Buses Table

The Buses table stores information about buses operated by registered operators.

Column	Data Type	Constraint	Description
BusID	INT	Primary Key, Identity	Unique bus identifier
OperatorID	INT	Foreign Key, NOT NULL	References Operators.OperatorID
BusNumber	NVARCHAR(30)	Unique, NOT NULL	Unique bus number
BusType	NVARCHAR(20)	NOT NULL, CHECK	Bus type
TotalSeats	INT	NOT NULL, CHECK	Total number of seats

Column	Data Type	Constraint	Description
Status	NVARCHAR(20)	NOT NULL, CHECK, DEFAULT	Bus availability status
CreatedAt	DATETIME	NOT NULL, DEFAULT	Bus creation date

Bus Types

The database supports:

- AC
- Non-AC
- Sleeper

Status

A bus can be:

- Active
- Inactive

4.3.5 Routes Table

The Routes table stores the routes offered by the system.

Column	Data Type	Constraint	Description
RouteID	INT	Primary Key, Identity	Unique route identifier
FromLocation	NVARCHAR(100)	NOT NULL	Starting location
ToLocation	NVARCHAR(100)	NOT NULL	Destination

Column	Data Type	Constraint	Description
DistanceKM	DECIMAL(8,2)	NULL, CHECK	Route distance in kilometers
EstimatedTime	NVARCHAR(50)	NULL	Estimated travel duration
RouteFare	DECIMAL(10,2)	NOT NULL, CHECK	Base/default fare for the route
Status	NVARCHAR(20)	NOT NULL, CHECK, DEFAULT	Route status

Constraints

DistanceKM, when provided, must be greater than zero.

RouteFare cannot be negative.

Status can be:

- Active
- Inactive

4.3.6 Schedules Table

The Schedules table connects buses and routes and defines when a particular bus operates on a route.

Column	Data Type	Constraint	Description
ScheduleID	INT	Primary Key, Identity	Unique schedule identifier

Column	Data Type	Constraint	Description
BusID	INT	Foreign Key, NOT NULL	References Buses.BusID
RouteID	INT	Foreign Key, NOT NULL	References Routes.RouteID
TravelDate	DATE	NOT NULL	Date of travel
DepartureTime	TIME(7)	NOT NULL	Scheduled departure time
ArrivalTime	TIME(7)	NOT NULL	Scheduled arrival time
Fare	DECIMAL(10,2)	NOT NULL, CHECK	Fare applicable to this schedule
Status	NVARCHAR(20)	NOT NULL, CHECK, DEFAULT	Schedule status
CreatedAt	DATETIME	NOT NULL, DEFAULT	Schedule creation date

Schedule Status

A schedule can be:

- Scheduled
- Completed
- Cancelled

The Schedules table is important because it represents a specific bus journey rather than simply a route.

4.3.7 Seats Table

The Seats table stores the physical seat layout of each bus.

Column	Data Type	Constraint	Description
SeatID	INT	Primary Key, Identity	Unique seat identifier
BusID	INT	Foreign Key, NOT NULL	References Buses.BusID
SeatNumber	NVARCHAR(10)	NOT NULL	Displayed seat number
SeatType	NVARCHAR(20)	NOT NULL, CHECK, DEFAULT	Type of seat
RowNumber	INT	NOT NULL, CHECK	Physical row position
ColumnNumber	INT	NOT NULL, CHECK	Physical column position
Deck	NVARCHAR(20)	NOT NULL, CHECK, DEFAULT	Upper or Lower deck
Status	NVARCHAR(20)	NOT NULL, CHECK, DEFAULT	Seat availability status

Seat Types

The database supports:

- Regular
- VIP

- Sleeper

Deck Types

The database supports:

- Upper
- Lower

Important Constraint

The combination of BusID and SeatNumber must be unique.

This prevents the same bus from having duplicate seat numbers.

4.3.8 Bookings Table

The Bookings table represents a customer's ticket booking.

Column	Data Type	Constraint	Description
BookingID	INT	Primary Key, Identity	Unique booking identifier
CustomerID	INT	Foreign Key, NOT NULL	References Customers.CustomerID
ScheduleID	INT	Foreign Key, NOT NULL	References Schedules.ScheduleID
BookingDate	DATETIME	NOT NULL, DEFAULT	Date and time of booking

Column	Data Type	Constraint	Description
TotalSeats	INT	NOT NULL, CHECK	Number of seats booked
TotalAmount	DECIMAL(10,2)	NOT NULL, CHECK	Total booking amount
BookingStatus	NVARCHAR(20)	NOT NULL, DEFAULT, CHECK	Booking status
PaymentStatus	NVARCHAR(20)	NOT NULL, DEFAULT, CHECK	Payment status

Booking Status

Allowed values are:

- Confirmed
- Completed
- Cancelled

Payment Status

Allowed values are:

- Pending
- Paid
- Refunded

TotalSeats must be greater than zero and TotalAmount cannot be negative.

4.3.9 BookingSeats Table

BookingSeats is an associative table used to connect bookings with seats.

It resolves the many-to-many relationship between bookings and seats.

Column	Data Type	Constraint	Description
BookingSeatID	INT	Primary Key, Identity	Unique booking-seat record
BookingID	INT	Foreign Key, NOT NULL	References Bookings.BookingID
SeatID	INT	Foreign Key, NOT NULL	References Seats.SeatID

Unique Constraint

The combination of:

BookingID + SeatID

must be unique.

This prevents the same seat from being inserted more than once for the same booking.

4.3.10 Payments Table

The Payments table stores payment information associated with customer bookings.

Column	Data Type	Constraint	Description
PaymentID	INT	Primary Key, Identity	Unique payment identifier
BookingID	INT	Foreign Key, NOT NULL	References Bookings.BookingID
Amount	DECIMAL(10,2)	NOT NULL	Amount paid
PaymentMethod	VARCHAR(30)	NOT NULL	Payment method
PaymentStatus	VARCHAR(20)	NOT NULL, DEFAULT	Payment status
TransactionID	VARCHAR(100)	NULL	External/internal transaction identifier
PaymentDate	DATETIME	NULL	Date and time of payment
CreatedAt	DATETIME	NOT NULL, DEFAULT	Payment record creation date

The table allows payment information to be linked directly to a booking.

4.3.11 OperatorPayments Table

The OperatorPayments table records payments made by operators toward their platform commission obligations.

Column	Data Type	Constraint	Description
OperatorPaymentID	INT	Primary Key, Identity	Unique operator payment identifier
OperatorID	INT	Foreign Key, NOT NULL	References Operators.OperatorID
Amount	DECIMAL(10,2)	NOT NULL	Commission amount paid/due
PaymentMethod	VARCHAR(30)	NULL	Payment method
TransactionID	VARCHAR(100)	NULL	Payment transaction identifier
PaymentStatus	VARCHAR(20)	NOT NULL, DEFAULT	Payment status
PaymentDate	DATETIME	NULL	Actual payment date
DueDate	DATE	NULL	Commission payment due date
Notes	VARCHAR(500)	NULL	Additional payment information
CreatedAt	DATETIME	NOT NULL, DEFAULT	Record creation date

This table enables the Super Admin to monitor operator commission payments.

4.3.12 Reviews Table

The Reviews table stores customer feedback for completed bookings.

Column	Data Type	Constraint	Description
ReviewID	INT	Primary Key, Identity	Unique review identifier
BookingID	INT	Foreign Key, Unique, NOT NULL	References Bookings.BookingID
CustomerID	INT	Foreign Key, NOT NULL	References Customers.CustomerID
Rating	INT	NOT NULL, CHECK	Customer rating from 1 to 5
Comment	VARCHAR(500)	NULL	Customer review/comment
ReviewDate	DATETIME	NOT NULL, DEFAULT	Date and time of review

Rating Constraint

The Rating value must be between **1 and 5**.

The unique constraint on BookingID means that a booking can have at most one review.

4.3.13 Notifications Table

The Notifications table stores system notifications intended for customers or operators.

Column	Data Type	Constraint	Description
NotificationID	INT	Primary Key, Identity	Unique notification identifier
CustomerID	INT	Foreign Key, NULL	Target customer
OperatorID	INT	Foreign Key, NULL	Target operator
Title	VARCHAR(150) NOT NULL		Notification title
Message	VARCHAR(500) NOT NULL		Notification content
NotificationType	VARCHAR(50)	NULL	Type/category of notification
IsRead	BIT	NOT NULL, DEFAULT	Indicates whether notification was read
CreatedAt	DATETIME	NOT NULL, DEFAULT	Notification creation date

The nullable CustomerID and OperatorID allow notifications to be associated with either a customer or an operator.

4.4 Primary Keys and Foreign Keys

4.4.1 Primary Keys

A primary key uniquely identifies every record in a table.

Table	Primary Key
Users	UserID
Customers	CustomerID
Operators	OperatorID
Buses	BusID
Routes	RouteID
Schedules	ScheduleID
Seats	SeatID
Bookings	BookingID
BookingSeats	BookingSeatID
Payments	PaymentID
OperatorPayments	OperatorPaymentID
Reviews	ReviewID
Notifications	NotificationID

All primary keys in the database use IDENTITY(1,1), allowing SQL Server to automatically generate unique numeric identifiers.

4.4.2 Foreign Keys

Foreign keys establish relationships between related tables.

Child Table	Foreign Key	Parent Table	Parent Key
Customers	UserID	Users	UserID
Operators	UserID	Users	UserID

Child Table	Foreign Key	Parent Table	Parent Key
Buses	OperatorID	Operators	OperatorID
Seats	BusID	Buses	BusID
Schedules	BusID	Buses	BusID
Schedules	RouteID	Routes	RouteID
Bookings	CustomerID	Customers	CustomerID
Bookings	ScheduleID	Schedules	ScheduleID
BookingSeats	BookingID	Bookings	BookingID
BookingSeats	SeatID	Seats	SeatID
Payments	BookingID	Bookings	BookingID
OperatorPayments	OperatorID	Operators	OperatorID
Reviews	BookingID	Bookings	BookingID
Reviews	CustomerID	Customers	CustomerID
Notifications	CustomerID	Customers	CustomerID
Notifications	OperatorID	Operators	OperatorID

4.4.3 Important One-to-One Relationships

Two particularly important relationships are:

Users → Customers

Customers.UserID references Users.UserID and has a unique constraint.

Therefore:

One User → Zero or One Customer Profile

Users → Operators

Operators.UserID references Users.UserID and has a unique constraint.

Therefore:

One User → Zero or One Operator Profile

These relationships allow general account information to remain in the Users table while role-specific information is stored separately.

4.5 Normalization

Normalization is the process of organizing data in a relational database to reduce unnecessary duplication, prevent update anomalies, and maintain data integrity.

The database design follows the principles of normalization, particularly **First Normal Form (1NF)**, **Second Normal Form (2NF)**, and **Third Normal Form (3NF)** for its core entities.

4.5.1 First Normal Form — 1NF

A relation satisfies First Normal Form when:

1. Each column contains atomic values.
2. There are no repeating groups.
3. Each record can be uniquely identified.

The Bus Ticket Management System follows 1NF because individual attributes are stored separately.

For example, the Users table does not store multiple phone numbers or multiple roles in a single field.

Instead, each user has separate attributes:

UserID

Username

Password

FullName

Email

Phone

Role

Status

CreatedAt

Similarly, multiple seats selected for one booking are not stored as:

Seat1, Seat2, Seat3, Seat4

inside the Bookings table.

Instead, the database uses the separate BookingSeats table:

BookingID | SeatID

This is an important example of achieving 1NF and properly representing multiple seat selections.

4.5.2 Second Normal Form — 2NF

A relation satisfies Second Normal Form when:

1. It is already in 1NF.
2. Every non-key attribute depends on the entire primary key.
3. There are no partial dependencies on part of a composite key.

Most tables in this database use a single-column primary key such as:

- UserID
- CustomerID
- OperatorID
- BusID
- BookingID
- PaymentID

Therefore, partial dependency is not possible in those tables.

The BookingSeats table is particularly important because it represents the relationship between bookings and seats.

Conceptually, the relationship is:

Booking + Seat → BookingSeat relationship

The table also has a unique constraint on:

BookingID + SeatID

which prevents duplicate booking-seat combinations.

Therefore, seat-selection information is separated from the main booking entity rather than storing repeating seat columns in Bookings.

4.5.3 Third Normal Form — 3NF

A relation satisfies Third Normal Form when:

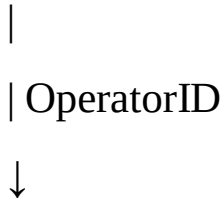
1. It is already in 2NF.
2. Non-key attributes depend on the key.
3. Non-key attributes do not depend on other non-key attributes.

The database separates independent entities into their own tables to reduce transitive dependencies.

For example, operator information is not repeatedly stored in the Buses table.

Instead:

Buses



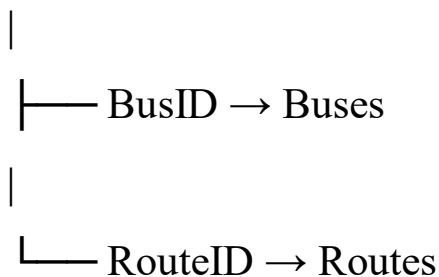
Operators

The bus table stores only the OperatorID reference.

Similarly, schedule information does not duplicate the complete bus and route information.

Instead:

Schedules



This avoids storing values such as:

CompanyName

BusNumber

FromLocation

ToLocation

repeatedly inside every schedule record.

Customer information is also separated:

Users



Customers



Bookings

Therefore, the customer's account information does not need to be repeated inside every booking.

4.5.4 Functional Dependencies

Functional dependency describes a relationship where one attribute determines another attribute.

The database contains several important functional dependencies.

Users

$\text{UserID} \rightarrow \text{Username, Password, FullName, Email, Phone, Role, Status, CreatedAt}$

A UserID uniquely determines the information belonging to that user.

Because Username is also unique:

$\text{Username} \rightarrow \text{UserID}$

within the database.

Customers

$\text{CustomerID} \rightarrow \text{UserID, Address, CreatedAt}$

Operators

$\text{OperatorID} \rightarrow \text{UserID, CompanyName, CompanyAddress,}$

$\text{LicenseNumber, CommissionRate, Status,}$

$\text{CreatedAt, Phone, Email}$

Buses

BusID → OperatorID, BusNumber, BusType,
TotalSeats, Status, CreatedAt

Routes

RouteID → FromLocation, ToLocation, DistanceKM,
EstimatedTime, RouteFare, Status

Schedules

ScheduleID → BusID, RouteID, TravelDate,
DepartureTime, ArrivalTime, Fare,
Status, CreatedAt

Bookings

BookingID → CustomerID, ScheduleID, BookingDate,
TotalSeats, TotalAmount,
BookingStatus, PaymentStatus

Payments

PaymentID → BookingID, Amount, PaymentMethod,
PaymentStatus, TransactionID,
PaymentDate, CreatedAt

These dependencies demonstrate that the attributes of each entity are primarily dependent on that entity's identifier.

4.5.5 Avoiding Redundancy

The database minimizes unnecessary duplication by separating different business entities.

For example, instead of storing operator details in every bus:

BusID

OperatorID

CompanyName

CompanyAddress

Phone

Email

the database stores:

Buses

BusID

OperatorID

BusNumber

BusType

...

and keeps company information in:

Operators

OperatorID

CompanyName

CompanyAddress

Phone

Email

...

Therefore, if an operator's phone number changes, it only needs to be updated in one location.

The same principle applies to:

- User information
- Customer information
- Operator information
- Bus information
- Route information
- Schedule information
- Booking information
- Payment information

4.5.6 Controlled Redundancy in the Implemented Database

Although the database follows normalization principles, a few fields are intentionally stored for operational convenience and application performance.

For example, `Bookings.PaymentStatus` stores the current payment state directly in the booking record, while detailed transaction information is stored in `Payments`.

Similarly, `Reviews.CustomerID` is stored together with `BookingID`, even though the customer can be identified through the related booking.

These fields can be considered controlled redundancy. They make common application operations and reporting easier, but they should be kept synchronized by the application logic.

The same design consideration applies to Routes.RouteFare and Schedules.Fare. The route contains a base/default fare, while the schedule stores the fare applicable to a particular scheduled journey. This allows a specific schedule to have a fare that may differ from the route's general fare.

Therefore, the database should be described as **normalized around its core entities with limited, intentional redundancy for operational requirements**, rather than claiming that every stored attribute is theoretically minimal.

4.5.7 Why 3NF is Important for This System

Applying normalization is particularly important for a bus ticket management system because the application contains many related entities.

Without normalization, the database could contain repeated information such as:

CustomerName

CustomerPhone

OperatorName

BusNumber

Route

DepartureTime

SeatNumber

PaymentInformation

inside every booking record.

This would create several problems:

Update Anomaly

If an operator changes their company information, many booking records would need to be updated.

Insert Anomaly

It would be difficult to add a new bus or route before a booking exists if all information were stored inside bookings.

Delete Anomaly

Deleting the last booking for a particular route could accidentally remove important route information.

By separating entities into related tables, the database avoids these problems.

For example:

Users



Customers



Bookings



BookingSeats



Seats



Buses



Operators

and:

Routes



Schedules



Bookings

allow each type of information to be maintained independently while still being connected through foreign keys.

4.6 Data Integrity Constraints

The database uses several types of constraints to maintain data integrity.

Primary Key Constraints

Every major table has a unique primary key.

Example:

Users.UserID

Buses.BusID

Bookings.BookingID

Payments.PaymentID

Foreign Key Constraints

Foreign keys prevent records from referencing nonexistent parent records.

For example:

Bookings.CustomerID



Customers.CustomerID

ensures that a booking must belong to a valid customer.

Unique Constraints

Unique constraints prevent duplicate values where uniqueness is required.

Examples include:

Users.Username

Buses.BusNumber

Customers.UserID

Operators.UserID

Seats(BusID, SeatNumber)

Reviews.BookingID

Check Constraints

Check constraints restrict values to valid business ranges.

Examples include:

CommissionRate >= 0 AND CommissionRate <= 100

Rating >= 1 AND Rating <= 5

TotalSeats > 0

TotalAmount >= 0

RouteFare >= 0

These constraints help prevent invalid data from being stored.

4.7 Default Values

The database also uses default values to automatically provide commonly required values.

Examples include:

Table	Column	Default
Users	Status	Active
Users	CreatedAt	GETDATE()
Operators	CommissionRate	10.00
Operators	Status	Active
Buses	Status	Active
Bookings	BookingDate	GETDATE()
Bookings	BookingStatus	Confirmed
Bookings	PaymentStatus	Pending
Payments	PaymentStatus	Pending
Routes	Status	Active
Schedules	Status	Scheduled
Seats	SeatType	Regular
Seats	Deck	Lower
Seats	Status	Active
Notifications	IsRead	0

Default values reduce the amount of repetitive input required from the application and ensure consistent initial states.

4.8 Overall Database Design Summary

The BusTicketManagementSystem database is designed as a relational database containing separate entities for authentication, customers, operators, transportation services, schedules, seats, bookings, payments, reviews, and notifications.

The design uses primary keys to uniquely identify records and foreign keys to establish relationships between entities. Unique and check constraints further protect the database from duplicate or invalid information.

The core structure separates independent entities and minimizes unnecessary duplication, following the principles of **1NF, 2NF, and 3NF**. The BookingSeats table resolves the relationship between bookings and seats, while the Users, Customers, and Operators structure separates common authentication information from role-specific information.

A small amount of intentional redundancy exists in fields such as booking/payment status and review/customer references to support practical application operations and reporting. These fields are controlled by the application logic and do not change the overall normalized structure of the core entities.

Overall, the database design provides a structured foundation for the Bus Ticket Management System and supports the complete workflow of the application, from user authentication and bus-service management to schedule selection, seat selection, ticket booking, payment processing, commission monitoring, and reporting.

Chapter 5 — SQL Queries

5.1 Introduction

The Bus Ticket Management System uses SQL Server to store, retrieve, update, and manage application data. SQL queries are used

throughout the system to implement authentication, bus management, route management, schedule management, seat selection, ticket booking, payment processing, operator commission monitoring, and Super Admin reporting.

Instead of presenting the complete database-generation script, this chapter presents the major SQL queries according to their application features.

For each major operation, the purpose, SQL query, explanation, and application usage are described.

5.2 Database Creation

Purpose

The database is created to store all information required by the Bus Ticket Management System.

SQL Query

```
CREATE DATABASE BusTicketManagementSystem;
```

Explanation

This statement creates the main SQL Server database used by the application.

The application then connects to the database using:

```
Data Source=(localdb)\MSSQLLocalDB;
```

```
Initial Catalog=BusTicketManagementSystem;
```

```
Integrated Security=True;
```

Expected Result / Application Use

After successful execution, a database named:

BusTicketManagementSystem

is created and becomes the main data source for the application.

5.3 User Management

User management is responsible for customer registration, authentication, account updates, and account status management.

5.3.1 Insert User

Purpose

Creates a new user account during registration.

SQL Query

```
INSERT INTO Users
```

```
(
```

```
    Username,
```

```
    Password,
```

```
    FullName,
```

```
    Email,
```

```
    Phone,
```

```
    Role,
```

```
    Status,
```

```
    CreatedAt
```

```
)
```

```
VALUES
```

```
(
```

```
    @Username,
```

```
@Password,  
@FullName,  
@Email,  
@Phone,  
@Role,  
'Active',  
GETDATE()  
);
```

Explanation

The query inserts the information provided during registration into the Users table.

Parameterized values such as @Username and @Password are used so that application input is not directly concatenated into the SQL statement.

Expected Result / Application Use

A new user account is created and can subsequently be used for authentication.

5.3.2 User Login

Purpose

Authenticates a user and identifies their role.

SQL Query

```
SELECT  
    UserID,  
    Username,
```

```
    FullName,  
    Role,  
    Status  
FROM Users  
WHERE Username = @Username  
    AND Password = @Password  
    AND Status = 'Active';
```

Explanation

The query searches for an active account whose username and password match the credentials supplied by the user.

The returned Role determines which part of the application the user can access.

Expected Result / Application Use

If a matching record exists, the application allows login and redirects the user to the appropriate role-specific interface.

For example:

SuperAdmin → SuperAdminDashBoard

Operator → AdminDashboard

Customer → CustomerDashboard

5.3.3 Update User

Purpose

Updates user information.

SQL Query

```
UPDATE Users
```

SET

FullName = @FullName,

Email = @Email,

Phone = @Phone

WHERE UserID = @UserID;

Explanation

The query updates the selected user's profile information using the unique UserID.

Expected Result / Application Use

The user's updated information is stored in the database.

5.3.4 Deactivate User

Purpose

Disables an account without physically deleting its database record.

SQL Query

UPDATE Users

SET Status = 'Inactive'

WHERE UserID = @UserID;

Explanation

Instead of deleting the user record, the system changes the account status to Inactive.

Expected Result / Application Use

The account remains in the database for historical integrity but cannot be used for normal active login.

5.4 Bus Management

Bus management is primarily performed by the Operator/Admin.

5.4.1 Add Bus

Purpose

Registers a new bus under an operator.

SQL Query

```
INSERT INTO Buses
```

```
(
```

```
    OperatorID,
```

```
    BusNumber,
```

```
    BusType,
```

```
    TotalSeats,
```

```
    Status,
```

```
    CreatedAt
```

```
)
```

```
VALUES
```

```
(
```

```
    @OperatorID,
```

```
    @BusNumber,
```

```
    @BusType,
```

```
    @TotalSeats,
```

```
    'Active',
```

```
    GETDATE()
```

);

Explanation

The query creates a bus record and associates it with an operator through OperatorID.

The BusType is restricted by the database to:

AC

Non-AC

Sleeper

Expected Result / Application Use

The newly registered bus appears in the operator's bus management interface.

5.4.2 Update Bus

Purpose

Updates existing bus information.

SQL Query

UPDATE Buses

SET

 BusNumber = @BusNumber,

 BusType = @BusType,

 TotalSeats = @TotalSeats,

 Status = @Status

WHERE BusID = @BusID

 AND OperatorID = @OperatorID;

Explanation

The OperatorID condition ensures that an operator can only update a bus belonging to that operator.

Expected Result / Application Use

The selected bus information is updated.

5.4.3 Delete / Deactivate Bus

Purpose

Makes a bus unavailable without permanently deleting its historical record.

SQL Query

UPDATE Buses

SET Status = 'Inactive'

WHERE BusID = @BusID

AND OperatorID = @OperatorID;

Explanation

The bus is marked inactive instead of being removed from the database.

This helps preserve relationships with historical schedules and bookings.

Expected Result / Application Use

The bus no longer appears as an active bus for normal operations.

5.4.4 Display Operator Buses

Purpose

Displays buses belonging to the currently logged-in operator.

SQL Query

```
SELECT
    BusID,
    BusNumber,
    BusType,
    TotalSeats,
    Status,
    CreatedAt
FROM Buses
WHERE OperatorID = @OperatorID
ORDER BY BusID DESC;
```

Expected Result / Application Use

The operator's buses are displayed in the **My Buses** interface.

5.5 Route Management

5.5.1 Add Route

Purpose

Creates a new travel route.

SQL Query

```
INSERT INTO Routes
(
    FromLocation,
```

```
ToLocation,  
DistanceKM,  
EstimatedTime,  
RouteFare,  
Status  
)  
VALUES  
(  
    @FromLocation,  
    @ToLocation,  
    @DistanceKM,  
    @EstimatedTime,  
    @RouteFare,  
    'Active'  
);
```

Explanation

The query stores the origin, destination, distance, estimated travel time, and base route fare.

Expected Result / Application Use

The route becomes available for schedule creation.

5.5.2 Update Route

Purpose

Updates route information.

SQL Query

UPDATE Routes

SET

FromLocation = @FromLocation,

ToLocation = @ToLocation,

DistanceKM = @DistanceKM,

EstimatedTime = @EstimatedTime,

RouteFare = @RouteFare,

Status = @Status

WHERE RouteID = @RouteID;

Expected Result / Application Use

The selected route information is updated in the database.

5.5.3 Display Active Routes

Purpose

Retrieves routes available for scheduling or searching.

SQL Query

SELECT

RouteID,

FromLocation,

ToLocation,

DistanceKM,

EstimatedTime,

RouteFare,

Status

FROM Routes

WHERE Status = 'Active'

ORDER BY FromLocation, ToLocation;

Expected Result / Application Use

Active routes can be displayed in route-related ComboBoxes and search interfaces.

5.6 Schedule Management

Schedules connect a specific bus with a specific route and travel date/time.

5.6.1 Add Schedule

Purpose

Creates a new bus travel schedule.

SQL Query

INSERT INTO Schedules

(

 BusID,

 RouteID,

 TravelDate,

 DepartureTime,

 ArrivalTime,

 Fare,

```
Status,  
CreatedAt  
)  
VALUES  
(  
    @BusID,  
    @RouteID,  
    @TravelDate,  
    @DepartureTime,  
    @ArrivalTime,  
    @Fare,  
    'Scheduled',  
    GETDATE()  
);
```

Explanation

A schedule associates a bus and route with a particular travel date, departure time, arrival time, and fare.

Expected Result / Application Use

The new schedule becomes available for customers to search and book.

5.6.2 Display Operator Schedules

Purpose

Displays schedules associated with the current operator's buses.

SQL Query

SELECT

s.ScheduleID,

s.TravelDate,

CONVERT(VARCHAR(5), s.DepartureTime, 108) AS
DepartureTime,

CONVERT(VARCHAR(5), s.ArrivalTime, 108) AS ArrivalTime,

s.Fare,

s.Status,

b.BusNumber,

r.FromLocation,

r.ToLocation

FROM Schedules s

INNER JOIN Buses b

ON s.BusID = b.BusID

INNER JOIN Routes r

ON s.RouteID = r.RouteID

WHERE b.OperatorID = @OperatorID

ORDER BY s.TravelDate, s.DepartureTime;

Explanation

The query joins Schedules, Buses, and Routes to display meaningful schedule information instead of showing only foreign-key IDs.

CONVERT(VARCHAR(5), ..., 108) is used to display SQL Server TIME values in HH:mm format.

Expected Result / Application Use

The Operator/Admin can view schedules in the **Schedules** form.

5.6.3 Update Schedule Status

Purpose

Changes a schedule's status.

SQL Query

UPDATE Schedules

SET Status = @Status

WHERE ScheduleID = @ScheduleID;

Expected Result / Application Use

A schedule can be marked as:

Scheduled

Completed

Cancelled

5.7 Seat Management

Seats are associated with individual buses.

5.7.1 Insert Bus Seats

Purpose

Creates the physical seat records for a bus.

SQL Query

INSERT INTO Seats

(

```
BusID,  
SeatNumber,  
SeatType,  
RowNumber,  
ColumnNumber,  
Deck,  
Status  
)  
VALUES  
(  
    @BusID,  
    @SeatNumber,  
    @SeatType,  
    @RowNumber,  
    @ColumnNumber,  
    @Deck,  
    'Active'  
);
```

Explanation

Each physical seat is stored as an individual record.

The row and column values allow the application to represent different seat arrangements.

This is useful because AC, Non-AC, and Sleeper buses may have different physical layouts.

Expected Result / Application Use

The customer-facing booking interface can retrieve and display the seats associated with the selected bus.

5.7.2 Display Seats for a Bus

Purpose

Retrieves seats belonging to a selected bus.

SQL Query

```
SELECT
```

```
    SeatID,
```

```
    SeatNumber,
```

```
    SeatType,
```

```
    RowNumber,
```

```
    ColumnNumber,
```

```
    Deck,
```

```
    Status
```

```
FROM Seats
```

```
WHERE BusID = @BusID
```

```
ORDER BY RowNumber, ColumnNumber;
```

Expected Result / Application Use

The application uses the returned data to create the visual seat-selection interface.

5.8 Bus Search

Bus search is one of the main customer features.

5.8.1 Search Available Buses

Purpose

Finds available schedules based on origin, destination, and travel date.

SQL Query

```
SELECT
    s.ScheduleID,
    b.BusID,
    b.BusNumber,
    b.BusType,
    r.RouteID,
    r.FromLocation,
    r.ToLocation,
    s.TravelDate,
    s.DepartureTime,
    s.ArrivalTime,
    s.Fare,
    s.Status
FROM Schedules s
INNER JOIN Buses b
    ON s.BusID = b.BusID
INNER JOIN Routes r
    ON s.RouteID = r.RouteID
```

```
WHERE r.FromLocation = @FromLocation  
AND r.ToLocation = @ToLocation  
AND s.TravelDate = @TravelDate  
AND b.Status = 'Active'  
AND r.Status = 'Active'  
AND s.Status = 'Scheduled';
```

Explanation

The query joins schedules with their associated buses and routes.

It filters the results using:

- Starting location
- Destination
- Travel date
- Active bus status
- Active route status
- Scheduled status

Expected Result / Application Use

The customer sees matching buses and schedules in the bus-search interface.

5.9 Booking

5.9.1 Create Booking

Purpose

Creates a new booking record for a customer.

SQL Query

```
INSERT INTO Bookings
```

```
(  
    CustomerID,  
    ScheduleID,  
    BookingDate,  
    TotalSeats,  
    TotalAmount,  
    BookingStatus,  
    PaymentStatus  
)
```

```
VALUES
```

```
(  
    @CustomerID,  
    @ScheduleID,  
    GETDATE(),  
    @TotalSeats,  
    @TotalAmount,  
    'Confirmed',  
    'Pending'  
);
```

Explanation

The booking stores the customer, selected schedule, number of seats, total amount, booking status, and payment status.

The payment status initially remains Pending until payment is completed.

Expected Result / Application Use

A booking record is created and receives a generated BookingID.

5.9.2 Retrieve Booking

Purpose

Retrieves booking information for a customer.

SQL Query

SELECT

b.BookingID,
b.BookingDate,
b.TotalSeats,
b.TotalAmount,
b.BookingStatus,
b.PaymentStatus,
s.TravelDate,
s.DepartureTime,
s.ArrivalTime,
bus.BusNumber,
r.FromLocation,
r.ToLocation

FROM Bookings b

INNER JOIN Schedules s

```
    ON b.ScheduleID = s.ScheduleID
INNER JOIN Buses bus
    ON s.BusID = bus.BusID
INNER JOIN Routes r
    ON s.RouteID = r.RouteID
WHERE b.CustomerID = @CustomerID
ORDER BY b.BookingDate DESC;
```

Expected Result / Application Use

The customer can view booking history and travel information.

5.10 Seat Booking

The booking process uses the BookingSeats table to associate selected seats with a booking.

5.10.1 Insert Selected Seat

Purpose

Associates a selected seat with a booking.

SQL Query

```
INSERT INTO BookingSeats
(
    BookingID,
    SeatID
)
VALUES
```

```
(  
    @BookingID,  
    @SeatID  
);
```

Explanation

Every selected seat receives a separate BookingSeats record.

For example, if a customer selects three seats:

BookingID = 25

SeatID = 4

SeatID = 5

SeatID = 6

three records are inserted.

Expected Result / Application Use

The selected seats become associated with the customer's booking.

5.10.2 Display Selected Seats

Purpose

Retrieves seats associated with a booking.

SQL Query

```
SELECT  
    bs.BookingSeatID,  
    s.SeatID,  
    s.SeatNumber,  
    s.SeatType,
```

```
s.RowNumber,  
s.ColumnNumber,  
s.Deck  
FROM BookingSeats bs  
INNER JOIN Seats s  
    ON bs.SeatID = s.SeatID  
WHERE bs.BookingID = @BookingID  
ORDER BY s.RowNumber, s.ColumnNumber;
```

Expected Result / Application Use

The customer can see which seats were selected for the booking.

5.11 Payment

5.11.1 Create Payment

Purpose

Records payment for a customer booking.

SQL Query

```
INSERT INTO Payments  
(  
    BookingID,  
    Amount,  
    PaymentMethod,  
    PaymentStatus,  
    TransactionID,
```

```
    PaymentDate,  
    CreatedAt  
)  
VALUES  
(  
    @BookingID,  
    @Amount,  
    @PaymentMethod,  
    'Paid',  
    @TransactionID,  
    GETDATE(),  
    GETDATE()  
);
```

Explanation

The payment record is connected to the booking through BookingID.

Expected Result / Application Use

A payment record is created for the booking.

5.11.2 Update Booking Payment Status

Purpose

Marks a booking as paid after successful payment.

SQL Query

```
UPDATE Bookings  
SET
```

PaymentStatus = 'Paid',

BookingStatus = 'Confirmed'

WHERE BookingID = @BookingID;

Expected Result / Application Use

The customer's booking changes from pending payment to paid/confirmed.

5.11.3 View Payment Information

Purpose

Retrieves payment information for a customer's booking.

SQL Query

SELECT

p.PaymentID,

p.BookingID,

p.Amount,

p.PaymentMethod,

p.PaymentStatus,

p.TransactionID,

p.PaymentDate

FROM Payments p

INNER JOIN Bookings b

ON p.BookingID = b.BookingID

WHERE b.CustomerID = @CustomerID

ORDER BY p.CreatedAt DESC;

Expected Result / Application Use

The customer can view their payment information.

5.12 Operator Commission

The system maintains a commission rate for each operator and records operator payments separately.

5.12.1 Calculate Operator Commission

Purpose

Calculates platform commission based on ticket sales.

SQL Query

```
SELECT
    o.OperatorID,
    o.CompanyName,
    o.CommissionRate,
    SUM(b.TotalAmount) AS TotalSales,
    SUM(b.TotalAmount * o.CommissionRate / 100.0)
    AS TotalCommission
FROM Operators o
INNER JOIN Buses bus
    ON o.OperatorID = bus.OperatorID
INNER JOIN Schedules s
    ON bus.BusID = s.BusID
INNER JOIN Bookings b
```

```
ON s.ScheduleID = b.ScheduleID
WHERE b.PaymentStatus = 'Paid'
GROUP BY
    o.OperatorID,
    o.CompanyName,
    o.CommissionRate;
```

Explanation

The query connects operators to their buses, schedules, and bookings.

For each operator, it calculates:

$\text{Commission} = \text{Total Sales} \times \text{Commission Rate} / 100$

Only paid bookings are included in the commission calculation.

Expected Result / Application Use

The Super Admin can monitor:

- Total sales
- Operator commission rate
- Total commission generated

5.12.2 Insert Operator Payment

Purpose

Records an operator's commission payment to the platform.

SQL Query

```
INSERT INTO OperatorPayments
(
    OperatorID,
```

```
Amount,  
PaymentMethod,  
TransactionID,  
PaymentStatus,  
PaymentDate,  
DueDate,  
Notes,  
CreatedAt  
)  
VALUES  
(  
    @OperatorID,  
    @Amount,  
    @PaymentMethod,  
    @TransactionID,  
    @PaymentStatus,  
    @PaymentDate,  
    @DueDate,  
    @Notes,  
    GETDATE()  
);
```

Expected Result / Application Use

The Super Admin can record and monitor commission payments made by operators.

5.12.3 View Operator Payment Status

Purpose

Displays operator commission payment records.

SQL Query

```
SELECT
    op.OperatorPaymentID,
    o.CompanyName,
    op.Amount,
    op.PaymentMethod,
    op.TransactionID,
    op.PaymentStatus,
    op.PaymentDate,
    op.DueDate,
    op.Notes
FROM OperatorPayments op
INNER JOIN Operators o
    ON op.OperatorID = o.OperatorID
ORDER BY
    op.DueDate,
    op.CreatedAt DESC;
```

Expected Result / Application Use

The Super Admin can identify payment records and determine whether operator payments are pending, paid, or otherwise recorded.

5.13 Super Admin Reports

The Super Admin dashboard requires aggregated information from multiple tables.

5.13.1 Total Operators

Purpose

Counts all registered operators.

SQL Query

```
SELECT COUNT(*) AS TotalOperators  
FROM Operators;
```

Expected Result / Application Use

The result is displayed in the Super Admin dashboard's operator statistics card.

5.13.2 Total Buses

Purpose

Counts registered buses.

SQL Query

```
SELECT COUNT(*) AS TotalBuses  
FROM Buses;
```

Expected Result / Application Use

Displays the total number of buses in the system.

5.13.3 Total Tickets Sold

Purpose

Counts seats/tickets associated with paid bookings.

SQL Query

```
SELECT  
  
    SUM(TotalSeats) AS TotalTicketsSold  
  
FROM Bookings  
  
WHERE PaymentStatus = 'Paid';
```

Expected Result / Application Use

Displays the total number of tickets/seats sold.

5.13.4 Total Sales

Purpose

Calculates total revenue from paid bookings.

SQL Query

```
SELECT  
  
    ISNULL(SUM(TotalAmount), 0) AS TotalSales  
  
FROM Bookings  
  
WHERE PaymentStatus = 'Paid';
```

Explanation

ISNULL ensures that the dashboard receives 0 instead of NULL when there are no paid bookings.

Expected Result / Application Use

Displays total ticket sales on the Super Admin dashboard.

5.13.5 Total Platform Commission

Purpose

Calculates the platform commission generated from paid ticket sales.

SQL Query

```
SELECT
    ISNULL(
        SUM(
            b.TotalAmount * o.CommissionRate / 100.0
        ), 0
    ) AS TotalCommission
FROM Bookings b
INNER JOIN Schedules s
    ON b.ScheduleID = s.ScheduleID
INNER JOIN Buses bus
    ON s.BusID = bus.BusID
INNER JOIN Operators o
    ON bus.OperatorID = o.OperatorID
WHERE b.PaymentStatus = 'Paid';
```

Expected Result / Application Use

The result is displayed as platform commission/revenue in the Super Admin dashboard.

5.13.6 Paid Operator Payments

Purpose

Calculates the total amount received from operators.

SQL Query

```
SELECT
```

```
    ISNULL(SUM(Amount), 0) AS ReceivedOperatorPayments
```

```
FROM OperatorPayments
```

```
WHERE PaymentStatus = 'Paid';
```

Expected Result / Application Use

Displays the total commission payment received from operators.

5.13.7 Pending Operator Payments**Purpose**

Calculates commission payment records that are still pending.

SQL Query

```
SELECT
```

```
    ISNULL(SUM(Amount), 0) AS PendingOperatorPayments
```

```
FROM OperatorPayments
```

```
WHERE PaymentStatus = 'Pending';
```

Expected Result / Application Use

Displays the amount of operator commission payments that have not yet been received.

5.13.8 Overdue Operator Payments**Purpose**

Identifies unpaid operator payments whose due date has passed.

SQL Query

```
SELECT
    op.OperatorPaymentID,
    o.CompanyName,
    op.Amount,
    op.DueDate,
    op.PaymentStatus
FROM OperatorPayments op
INNER JOIN Operators o
    ON op.OperatorID = o.OperatorID
WHERE op.PaymentStatus = 'Pending'
    AND op.DueDate < CAST(GETDATE() AS DATE)
ORDER BY op.DueDate;
```

Explanation

The query compares the payment due date with the current date.

A payment is considered overdue when:

PaymentStatus = Pending

AND

DueDate < Current Date

Expected Result / Application Use

The Super Admin can identify operators whose commission payments are overdue.

5.14 Customer Booking History

Purpose

Retrieves a customer's previous and current bookings.

SQL Query

```
SELECT
    b.BookingID,
    r.FromLocation,
    r.ToLocation,
    bus.BusNumber,
    s.TravelDate,
    s.DepartureTime,
    s.ArrivalTime,
    b.TotalSeats,
    b.TotalAmount,
    b.BookingStatus,
    b.PaymentStatus
FROM Bookings b
INNER JOIN Schedules s
    ON b.ScheduleID = s.ScheduleID
INNER JOIN Buses bus
    ON s.BusID = bus.BusID
INNER JOIN Routes r
    ON s.RouteID = r.RouteID
WHERE b.CustomerID = @CustomerID
```

ORDER BY b.BookingDate DESC;

Expected Result / Application Use

The customer can view their booking history and ticket-related information.

5.15 Review Management

Although reviews are not part of the primary booking flow, the database supports customer reviews.

Insert Review

Purpose

Stores customer feedback for a booking.

SQL Query

INSERT INTO Reviews

(

BookingID,

CustomerID,

Rating,

Comment,

ReviewDate

)

VALUES

(

@BookingID,

@CustomerID,

```
@Rating,  
@Comment,  
GETDATE()  
);
```

Explanation

The database restricts the rating to values between 1 and 5.

The unique constraint on BookingID prevents the same booking from receiving multiple review records.

Expected Result / Application Use

The customer's feedback is stored for future review/reporting purposes.

5.16 Notifications

Purpose

Creates a notification for a customer or operator.

SQL Query

```
INSERT INTO Notifications
```

```
(  
    CustomerID,  
    OperatorID,  
    Title,  
    Message,  
    NotificationType,  
    IsRead,
```

```

        CreatedAt
    )
VALUES
(
    @CustomerID,
    @OperatorID,
    @Title,
    @Message,
    @NotificationType,
    0,
    GETDATE()
);

```

Expected Result / Application Use

A notification record is created and can later be displayed to the intended user.

5.17 SQL Query Summary

The major SQL operations used by the Bus Ticket Management System can be summarized as follows:

FeatureMain Tables
UsedMain SQL
Operations

Database Creation

—

CREATE
DATABASE

User Management	Users	INSERT, SELECT, UPDATE
Customer Management	Users, Customers	INSERT, SELECT
Operator Management	Users, Operators	INSERT, SELECT, UPDATE
Bus Management	Buses	INSERT, SELECT, UPDATE
Route Management	Routes	INSERT, SELECT, UPDATE
Schedule Management	Schedules, Buses, Routes	INSERT, SELECT, UPDATE
Seat Management	Seats	INSERT, SELECT
Bus Search	Schedules, Buses, Routes	SELECT, JOIN, WHERE
Booking	Bookings	INSERT, SELECT
Seat Booking	BookingSeats, Seats	INSERT, SELECT

Customer Payment	Payments, Bookings	INSERT, UPDATE, SELECT
Operator Commission	Operators, Buses, Schedules, Bookings	SELECT, SUM, GROUP BY
Operator Payments	OperatorPayments, Operators	INSERT, SELECT
Super Admin Statistics	Multiple tables	COUNT, SUM
Booking History	Bookings, Schedules, Buses, Routes	SELECT, JOIN
Reviews	Reviews, Bookings, Customers	INSERT, SELECT
Notifications	Notifications, Customers, Operators	INSERT, SELECT

5.18 Conclusion

The SQL queries presented in this chapter demonstrate how the database supports the main functionality of the Bus Ticket Management System. Instead of treating the database as a collection of isolated tables, the application uses SQL joins, filtering, aggregation, insertion, updating, and transaction-related operations to implement complete business workflows.

The most important workflow is the customer ticket-booking process, where schedules are searched, seats are selected, a booking is created, selected seats are associated with the booking through BookingSeats, and payment information is stored in Payments.

The Super Admin functionality uses aggregate SQL queries to calculate system statistics such as total operators, total buses, tickets sold, sales, platform commission, and operator payment status.

Similarly, the Operator/Admin functionality uses queries involving buses, routes, and schedules to manage bus services, while customer-related queries provide searching, booking, seat selection, payment, and booking-history functionality.

Thus, the SQL layer forms the connection between the application's graphical user interface and the relational database, allowing the Bus Ticket Management System to perform its required operations while maintaining data consistency and integrity.

Chapter 6 — User Interface Design

Deliverable: D6

6.1 Introduction

The Bus Ticket Management System provides a role-based Windows Forms user interface designed to make the system simple and accessible for Super Admins, Operators, and Customers. The interface is divided into separate functional areas according to the responsibilities of each user role.

The main interface allows users to select their role and proceed to the appropriate authentication page. After successful authentication, each user is directed to a role-specific dashboard where the available functions are presented through buttons, panels, forms, and data tables.

The following sections present the major user interfaces of the system. Each figure includes a screenshot of the implemented interface together with a description of its purpose and functionality.

6.2 Main Role Selection Page

Figure 6.1 — Main Role Selection Page

Caption: *The main page allows the user to select their role before proceeding to the corresponding authentication interface.*

Explanation:

The Main Role Selection Page is the entry point of the Bus Ticket Management System. Users can select one of the available roles—**Super Admin, Operator/Admin, or Customer/Ticket Buyer**. After selecting a role, the user can continue to the appropriate login or registration interface. This design separates the functionalities of different user types and provides a clear starting point for system navigation.

Screenshot:

Insert screenshot of BusTicketMainPage here.

6.3 Super Admin Interfaces

Figure 6.2 — Super Admin Login

Caption: *The Super Admin Login interface provides secure access to administrative functions.*

Explanation:

The Super Admin Login page allows the system owner to enter a username and password before accessing the administrative dashboard. The interface also provides controls for continuing, cancelling the login process, changing login information, and showing or hiding the entered password.

Screenshot:

Insert screenshot of SuperAdmin here.

Figure 6.4 — Manage Operators

Caption: *The Manage Operators interface allows the Super Admin to view and manage operator information.*

Explanation:

The Manage Operators interface is used by the Super Admin to manage registered bus operators. The interface allows operator information to be displayed and provides management actions such as viewing, updating, and deleting operator records. This helps the Super Admin maintain control over the operators registered within the system.

Screenshot:

Insert screenshot of ManageOperators here.

Figure 6.3 — Super Admin Dashboard

Caption: *The Super Admin Dashboard provides an overview of the overall bus ticket management system.*

Explanation:

The Super Admin Dashboard is the central administrative interface of the system. It presents important system statistics through dashboard cards and provides navigation to administrative functions. The dashboard can display information such as the number of operators, buses, tickets sold, sales, platform commission, and operator payment status. The sidebar provides access to different management and reporting functions.

Screenshot:

Insert screenshot of SuperAdminDashBoard here.

Figure 6.5 — Payments & Reports

Caption: *The Payments and Reports interface provides Super Admins with information about platform commission and operator payments.*

Explanation:

The Payments and Reports interface is designed to monitor financial information generated by the system. It provides information related to ticket sales, platform commission, operator payments, and payment status. The interface helps the Super Admin identify received, pending, and overdue operator payments.

Screenshot:

Insert screenshot of PaymentsReports here.

6.4 Operator/Admin Interfaces

Figure 6.6 — Operator Dashboard

Caption: *The Operator Dashboard provides access to the main bus-service management functions.*

Explanation:

After successful authentication, an Operator is directed to the Operator Dashboard. The dashboard provides access to functions such as managing buses, routes, schedules, bookings, and payment reports. This interface acts as the central workspace for operators to manage their transportation services.

Screenshot:

Insert screenshot of AdminDashboard here.

Figure 6.7 — My Buses

Caption: *The My Buses interface allows operators to manage their registered buses.*

Explanation:

The My Buses interface allows an operator to add and manage buses associated with their account. Information such as bus number, bus type, seat capacity, and bus status can be maintained through this interface. This information is later used when creating schedules and providing buses for customers to book.

Screenshot:

Insert screenshot of MyBuses here.

Figure 6.8 — Routes

Caption: *The Routes interface allows operators to create and manage bus routes.*

Explanation:

The Routes interface is used to maintain route information such as departure location, destination, distance, estimated travel time, fare, and route status. These routes are subsequently associated with schedules so that customers can search for available buses based on their travel requirements.

Screenshot:

Insert screenshot of Routes here.

Figure 6.9 — Schedules

Caption: *The Schedules interface allows operators to create and manage bus travel schedules.*

Explanation:

The Schedules interface connects a bus with a particular route and travel date. Operators can specify departure time, arrival time, fare, and schedule status. The interface also displays existing schedules in a

structured table, making it easier for operators to monitor and update their available trips.

Screenshot:

Insert screenshot of Schedules here.

Figure 6.10 — Operator Bookings

Caption: *The Bookings interface allows operators to view customer booking information.*

Explanation:

The Bookings interface provides operators with access to booking information associated with their buses and schedules. It allows the operator to monitor customer reservations and relevant booking details generated through the ticket purchasing process.

Screenshot:

Insert screenshot of Bookings here.

6.5 Customer Interfaces

Figure 6.11 — Customer Login/Register

Caption: *The Customer Login and Registration interface allows customers to access the system or create a new account.*

Explanation:

The Customer Login/Register interface provides authentication facilities for customers. Existing customers can log in using their account credentials, while new users can register by providing the required information. Successful authentication redirects the customer to the Customer Dashboard.

Screenshot:

Insert screenshot of Customer_TicketBuyer_ here.

Figure 6.12 — Customer Dashboard

Caption: *The Customer Dashboard provides access to bus search, booking, and payment-related functions.*

Explanation:

The Customer Dashboard is the main workspace for ticket buyers. It provides navigation to the major customer functions, including searching for buses, booking tickets, making payments, and accessing relevant booking information. The dashboard is designed to provide a simple and user-friendly customer experience.

Screenshot:

Insert screenshot of CustomerDashboard here.

Figure 6.13 — Search Buses

Caption: *The Search Buses interface allows customers to find available buses according to their travel requirements.*

Explanation:

The Search Buses interface allows customers to search for available trips by providing information such as the departure location, destination, and travel date. The system displays matching schedules along with relevant bus and route information. Customers can select a suitable schedule and continue to the seat-selection process.

Screenshot:

Insert screenshot of SearchBuses here.

Figure 6.14 — Seat Selection

Caption: *The Seat Selection interface allows customers to visually select available seats before confirming a booking.*

Explanation:

The Seat Selection interface allows customers to choose their preferred seats from the available seats of the selected bus. The interface is designed to represent the bus seating arrangement so that customers can identify available and selected seats. The system supports different bus types and seating configurations, including regular and sleeper-style arrangements.

Screenshot:

Insert screenshot of your seat-selection interface here.

Figure 6.15 — Booking Confirmation

Caption: *The Booking Confirmation interface displays the selected travel and seat information before payment.*

Explanation:

The Booking Confirmation interface allows customers to review their selected schedule, seats, number of tickets, and total amount before completing the payment process. This provides the customer with an opportunity to verify the booking information before the transaction is finalized.

Screenshot:

Insert screenshot of the booking confirmation interface here.

Figure 6.16 — Payment

Caption: *The Payment interface allows customers to complete payment for their selected booking.*

Explanation:

The Payment interface is used to complete the ticket purchasing process. It displays the relevant booking information and payment amount and allows the customer to provide the required payment information. After successful payment, the booking and selected seats are recorded in the database and the booking status is updated accordingly.

Screenshot:

Insert screenshot of CustomerPayments here.

6.6 Overall UI Navigation

The user interface follows a **role-based navigation structure**. The user first enters through the Main Role Selection Page and selects either Super Admin, Operator/Admin, or Customer/Ticket Buyer. Each role is then directed to its respective authentication interface and dashboard. Super Admins can manage operators and monitor system-wide financial and operational information, while Operators manage buses, routes, schedules, and bookings. Customers use the system to search for buses, select seats, confirm bookings, and complete payments. This separation ensures that each user receives access only to the functions relevant to their responsibilities.

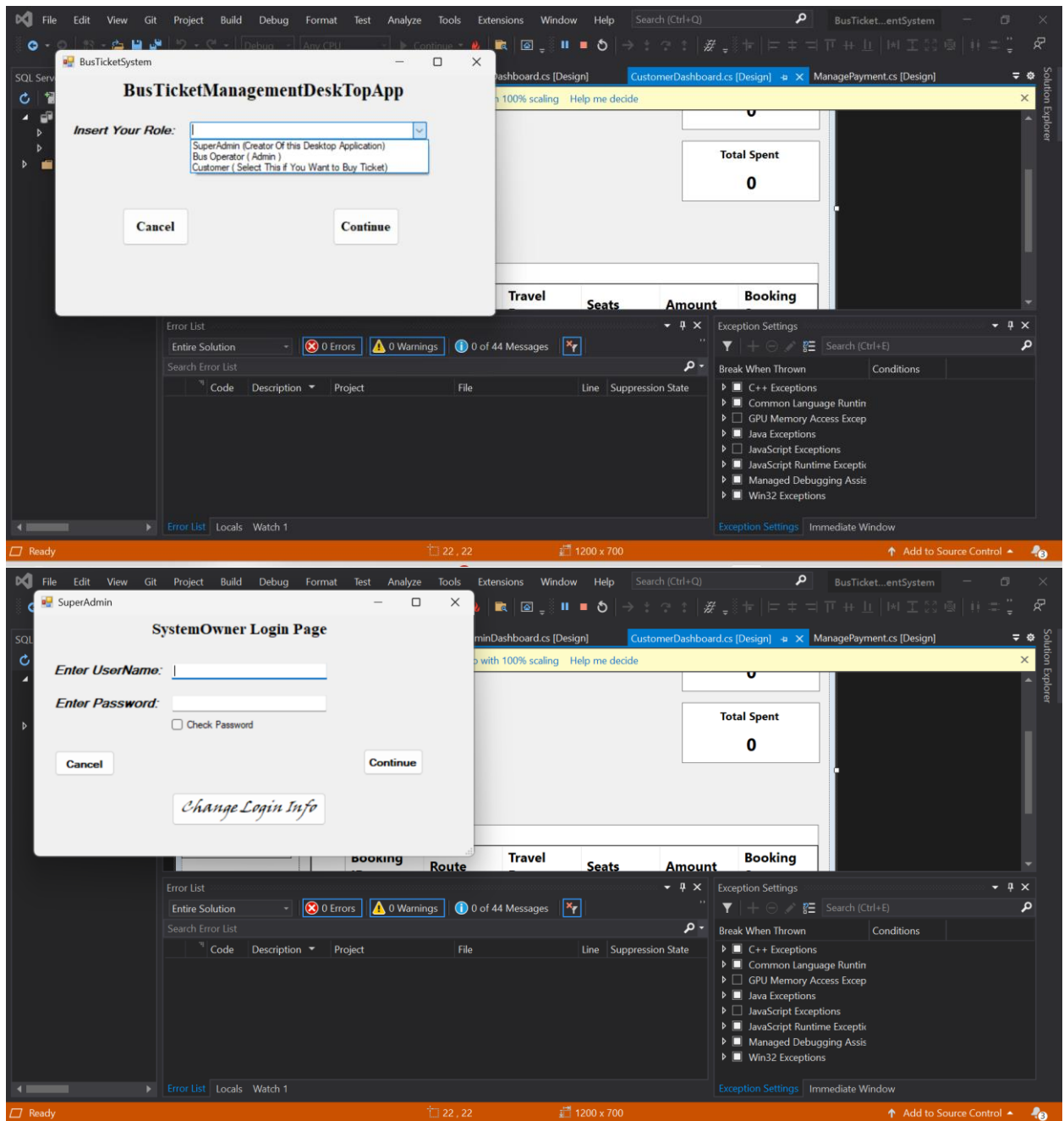
6.7 UI Design Considerations

The following design considerations were applied throughout the system:

- **Role-based navigation:** Different users are provided with different dashboards and functions.
- **Simple navigation:** Sidebar buttons and dashboard controls provide direct access to major functions.

- **Consistent layout:** Similar forms use consistent labels, buttons, tables, and input controls.
- **Data visibility:** Important information is displayed using tables and dashboard cards.
- **User feedback:** The application provides messages for successful operations, errors, and validation issues.
- **Input validation:** Forms validate required information before database operations are performed.
- **Seat visualization:** The customer booking interface provides a visual method for selecting available seats.
- **Financial transparency:** Administrative interfaces display sales, commission, and operator payment information.
- **Workflow-based design:** The customer interface follows a logical sequence from searching for a bus to selecting a seat, confirming the booking, and making payment.

These given screenshots are for SuperAdmin's UI one by one.



Dashboard

Operators

Buses

Reports

Settings

LogOut

Welcome BusGo Super Admin Panel
Manage operators, buses, payments and reports from here.

Total Operators

1

Active Operators

1

Pending Operator

0

Total Buses

1

Tickets Sold

4

Total Sales

₹ 3,400.

Manage Operators

Add Operator

Edit Operator

Delete Operator

Show Operator

Search Operator:

Search

	Operator ID	Operator Name	Company Name	Phone	Email	Status	Registration Date
▶	1	Rahim Bus Services	Rahim Express	01700000000	rahimexpress@gmail.com	Active	11-Sep-2026

Operator Name: Company Name: Phone: Email: Status:

Clear

BusGo

Super Admin

Dashboard

Operators

Buses

Reports

Settings

LogOut

Super Admin Dashboard

Welcome Super Admin

Manage Buses

Add Bus

Edit Bus

Delete Bus

Show Buses

Search:

Search

Bus ID	Bus Number	Operator Name	Route	Bus Type	Total Seats	Fare	Status
1	RE-101	Rahim Bus S...	Dhaka - Chitt...	AC	40	850.00	Active

Bus Number:

RE-101

Operator Name:

Rahim Bus Services

Route:

Dhaka - Chittagong

Bus Type:

AC

Total Seats:

40

Fare:

850.00

Status:

Active

Clear

BusGo

Super Admin

Dashboard

Operators

Buses

Reports

Settings

LogOut

Super Admin Dashboard

Welcome Super Admin

Manage Reports

Report Type:

Sales Report

Generate Report

Report ID	Report Name	Operator Name	Company Name	Total Sales	Commission	Total Payments	Report Date
1	Sales Report	Rahim Bus S...	Rahim Express	3,400.00	0.00	0.00	14-Sep-2026

Total Sales:

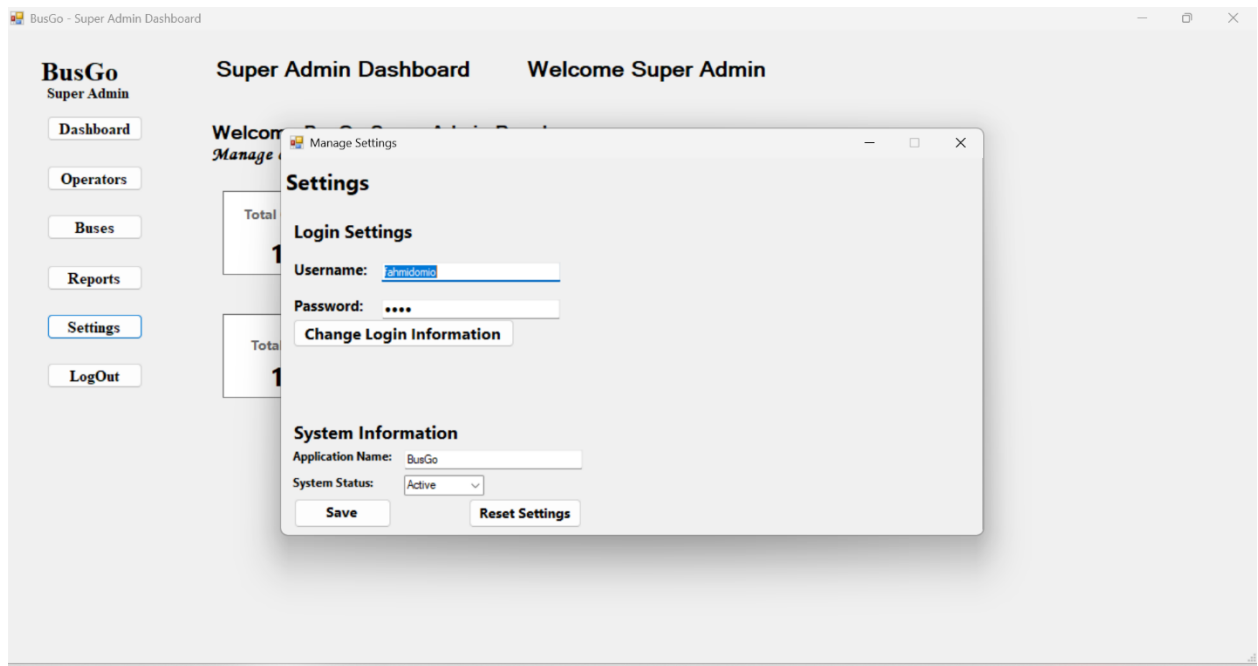
3,400.00

Total Commissions:

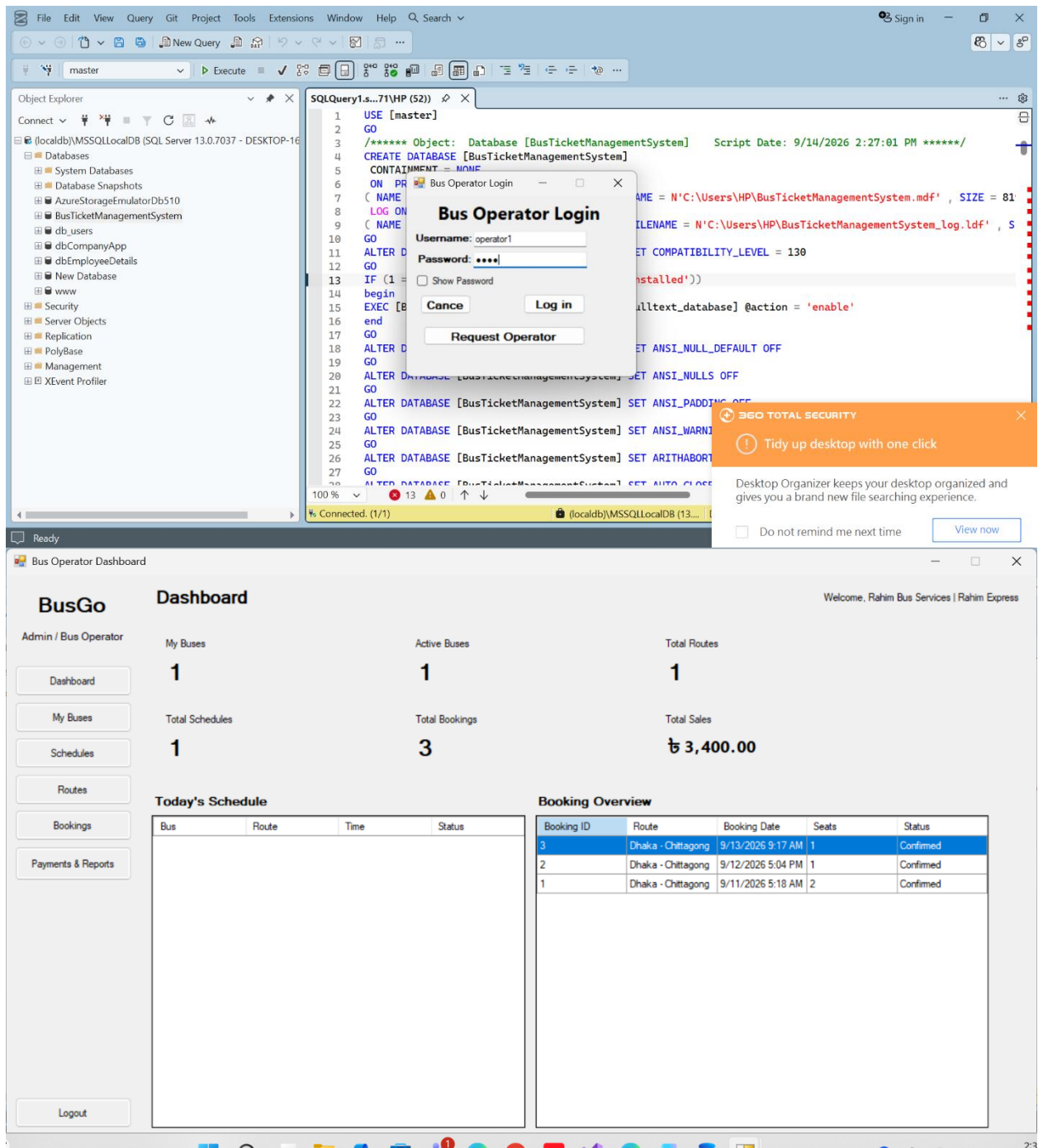
0.00

Total Payments:

0.00



These given screenshots are for Admin or Bus Operator given one by one ----



Bus Operator Dashboard

BusGo

Admin / Bus Operator

Dashboard

My Buses

Schedules

Routes

Bookings

Payments & Reports

Logout

Dashboard

Welcome, Rahim Bus Services | Rahim Express

My Buses

Active Buses

Total Routes

1

Total Schedules

1

Today's

Bus

My Buses

Add Bus

Edit Bus

Delete

Show Buses

Search:

Search

Bus ID	Bus Number	Route	Bus Type	Total Seats	Fare	Status
1	RE-101	Dhaka - Chittag...	AC	40	850.00	Active

Bus Number:

Route:

Bus Type:

Total Seats:

Fare:

Status:

Clear

Schedules

Add Schedule

Edit Schedule

Delete Schedule

Show Schedules

Search Schedule:

Search

Schedule ID	Departure Date	Departure Time	Arrival Time	Status	Bus Number	Route
1	9/20/2026	08:00	14:00	Scheduled	RE-101	Dhaka ? Chittag...

Bus Number:

Route:

Departure Time:

Arrival Time:

Status:

Departure Date:

Clear

81

S

UTF-8

DWS

81

S

UTF-8

DWS

Bus Operator Dashboard

BusGo

Admin / Bus Operator

Dashboard

My Buses

Schedules

Routes

Bookings

Payments & Reports

Logout

Dashboard

Welcome, Rahim Bus Services | Rahim Express

My Buses

Active Buses

Total Routes

1

1

1

Today's

Bus

Routes

Add Route

Edit Route

Delete Route

Show Routes

Search Route:

Search

	Route ID	From	To	Distance	Estimated Time	Fare	Status
▶	1	Dhaka	Chittagong	265.00	6 Hours	850.00	Active

From:

To:

Distance:

Estimated Time:

Fare:

Status:

Clear

Bookings

Show Bookings

View Details

Clear

Search Booking:

Search

	Booking ID	Customer Name	Bus Number	Route	Travel Date	Seats	Amount	Booking Status
▶	3	Kazi Fahmid	RE-101	Dhaka ? Chitt...	9/20/2026	1	850.00	Confirmed
	2	Kazi Fahmid	RE-101	Dhaka ? Chitt...	9/20/2026	1	850.00	Confirmed
	1	Kazi Fahmid	RE-101	Dhaka ? Chitt...	9/20/2026	2	1700.00	Confirmed

Customer Name:

Bus Number:

Route:

Travel Time:

9/14/2026

Seats:

Amount:

Booking Status:

Clear

81

S

UTF-8

DWS

Bus Operator Dashboard

Welcome, Rahim Bus Services | Rahim Express

BusGo Dashboard

Admin / Bus Operator

My Buses: 1, Active Buses: 1, Total Routes: 1

Today's Buses: 1

Buttons: Dashboard, My Buses, Schedules, Routes, Bookings, Payments & Reports, Logout

Payments and Reports

Search Payment: Report Type: Sales Report

Buttons: Show Payments, Generate Report, Clear

Payment ID	Payment Type	Amount	Payment Method	Payment Date	Payment Status
2	Customer Payment	850.00	Online	9/13/2026 9:17 AM	Paid
1	Customer Payment	1700.00	Card	9/11/2026 5:22 AM	Paid

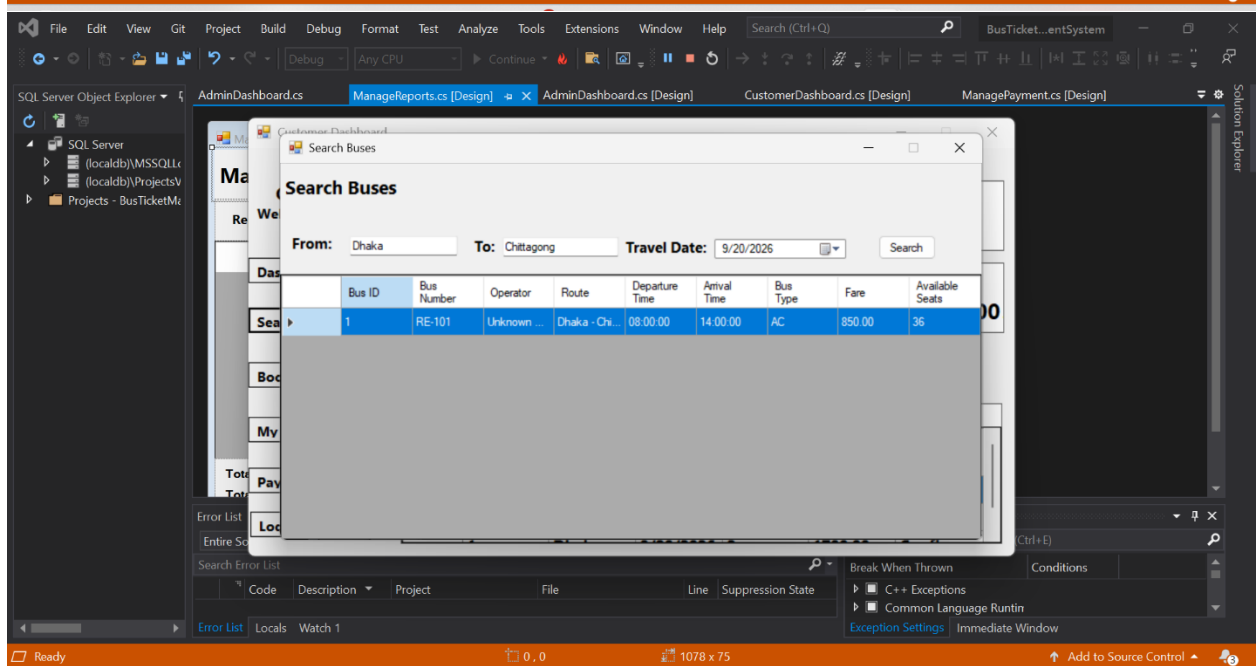
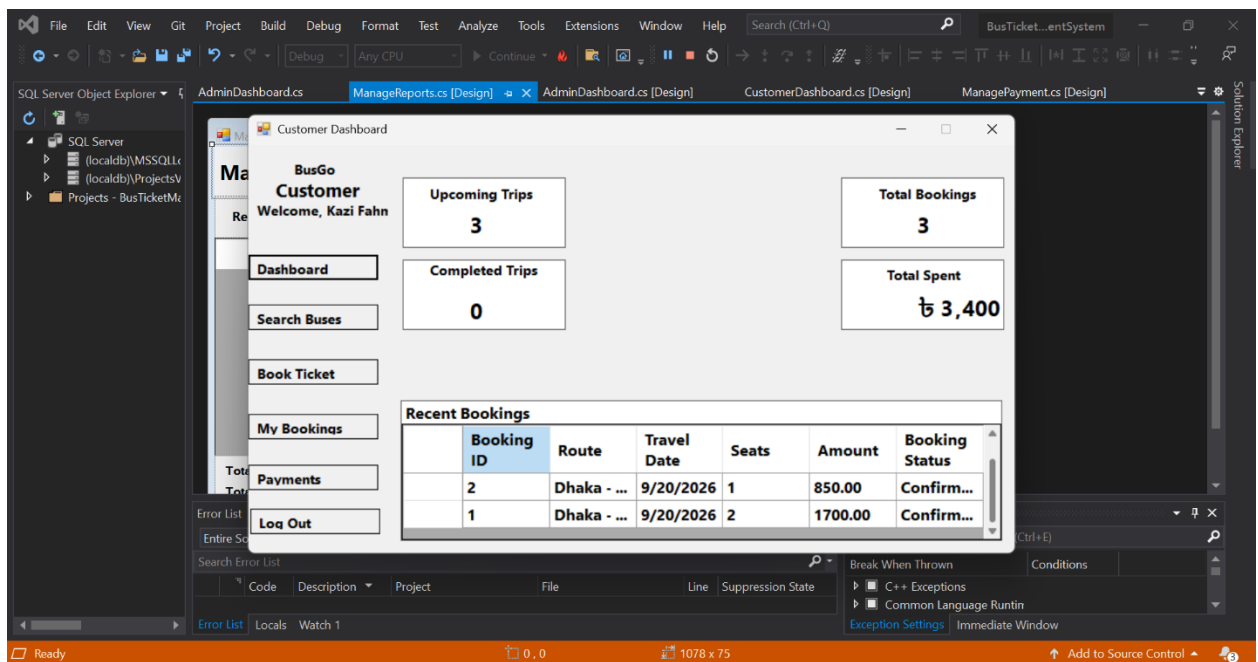
Status
Confirmed
Confirmed
Confirmed

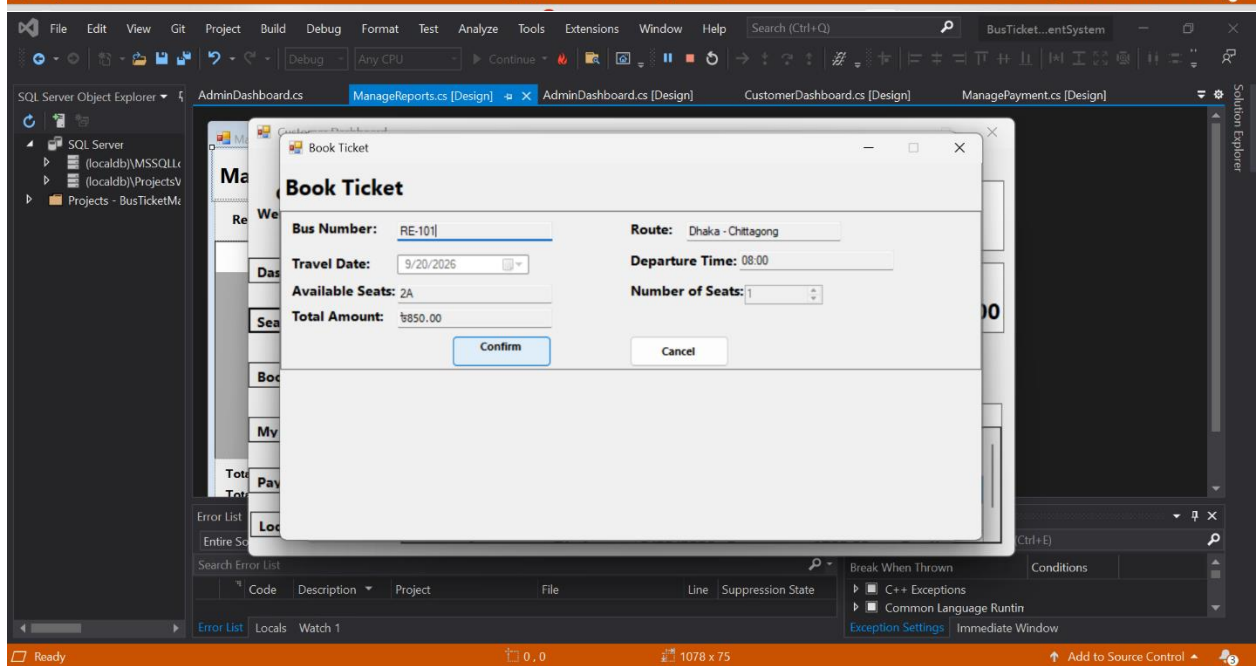
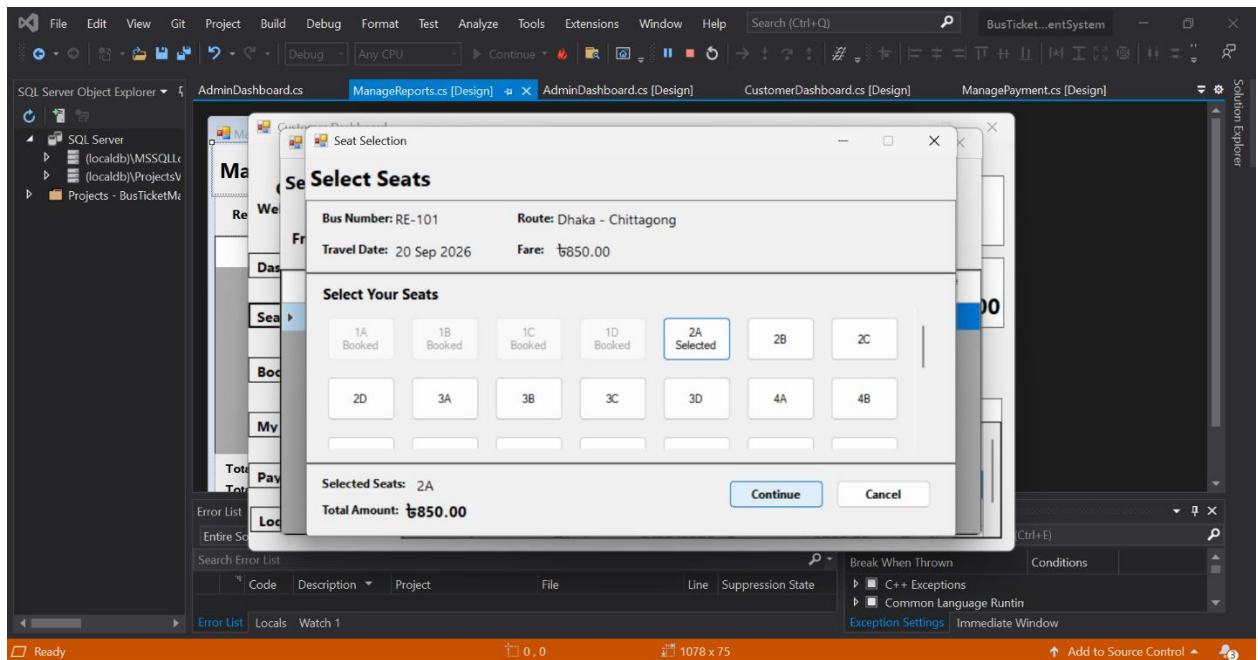
Total Sales
₹ 3,400.00

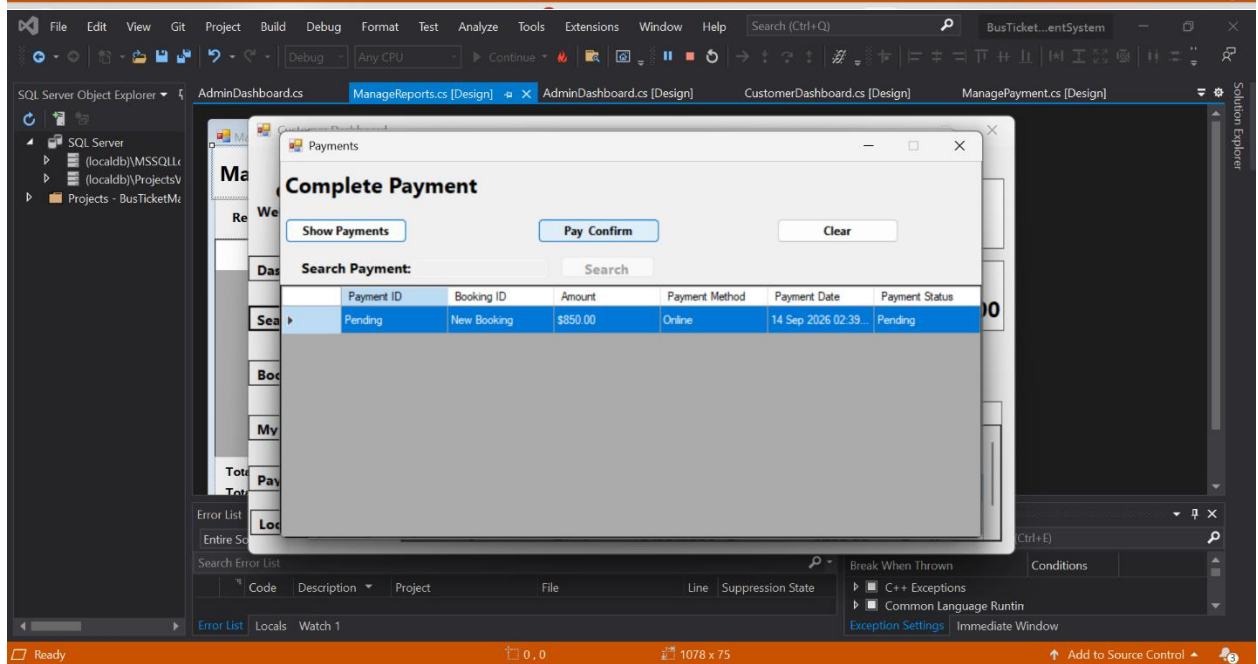
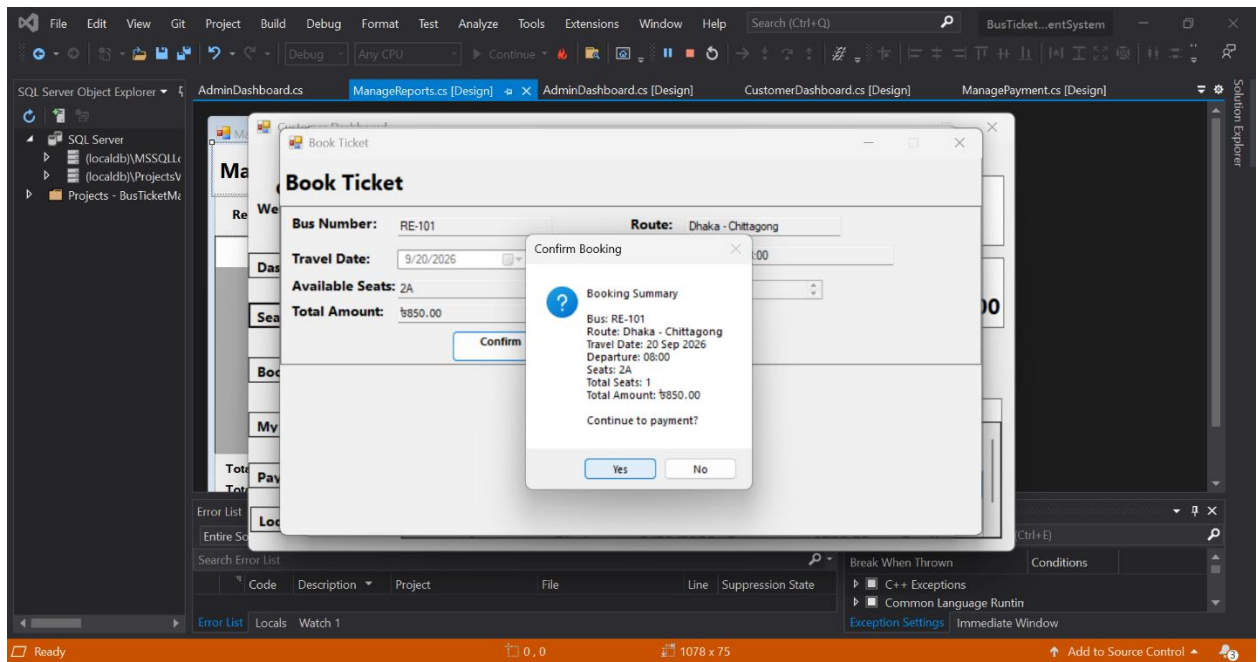
Total Commission
₹ 340.00

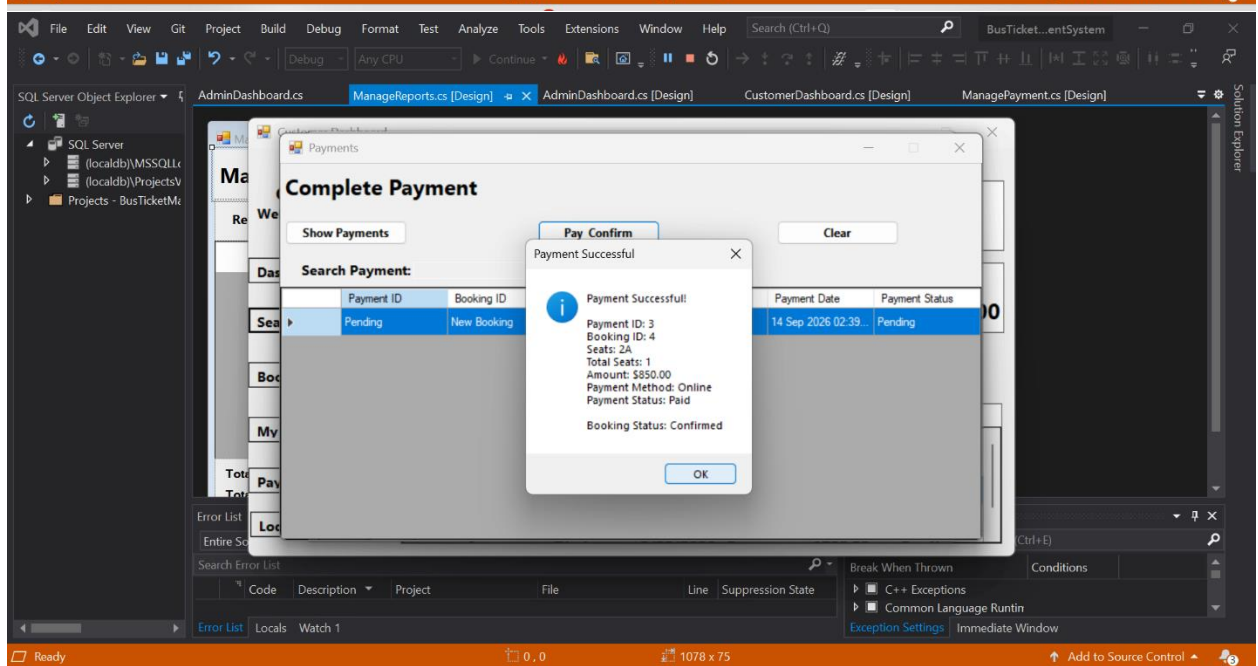
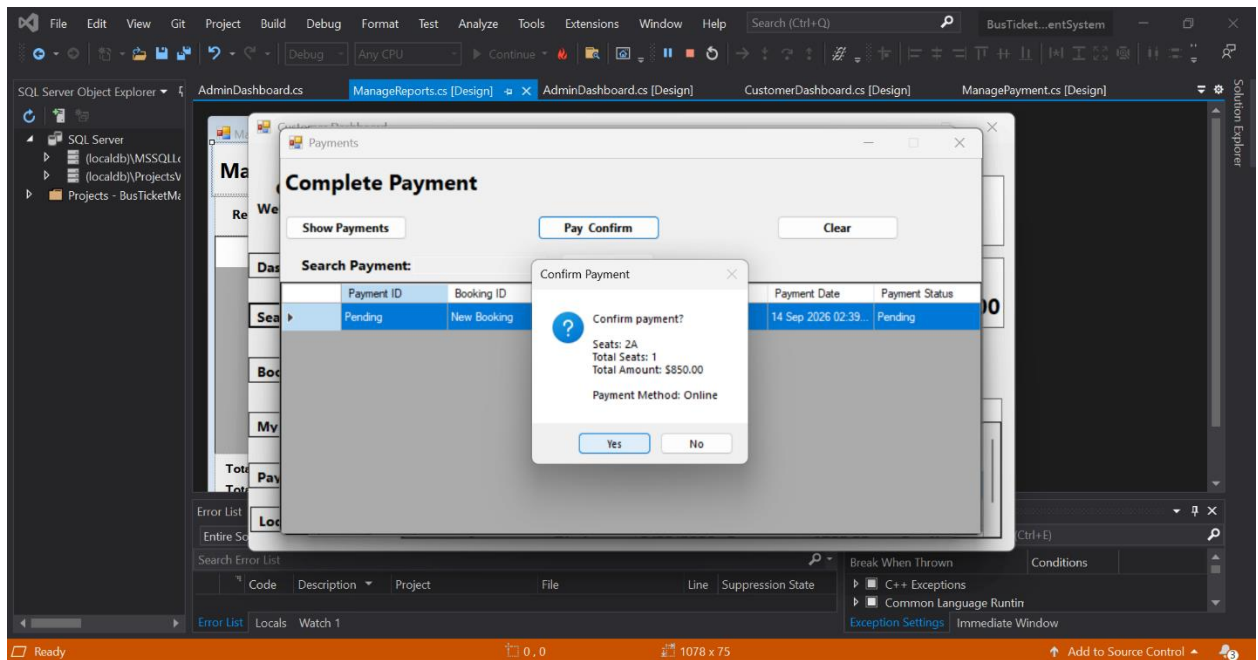
Total Payments
₹ 2,550.

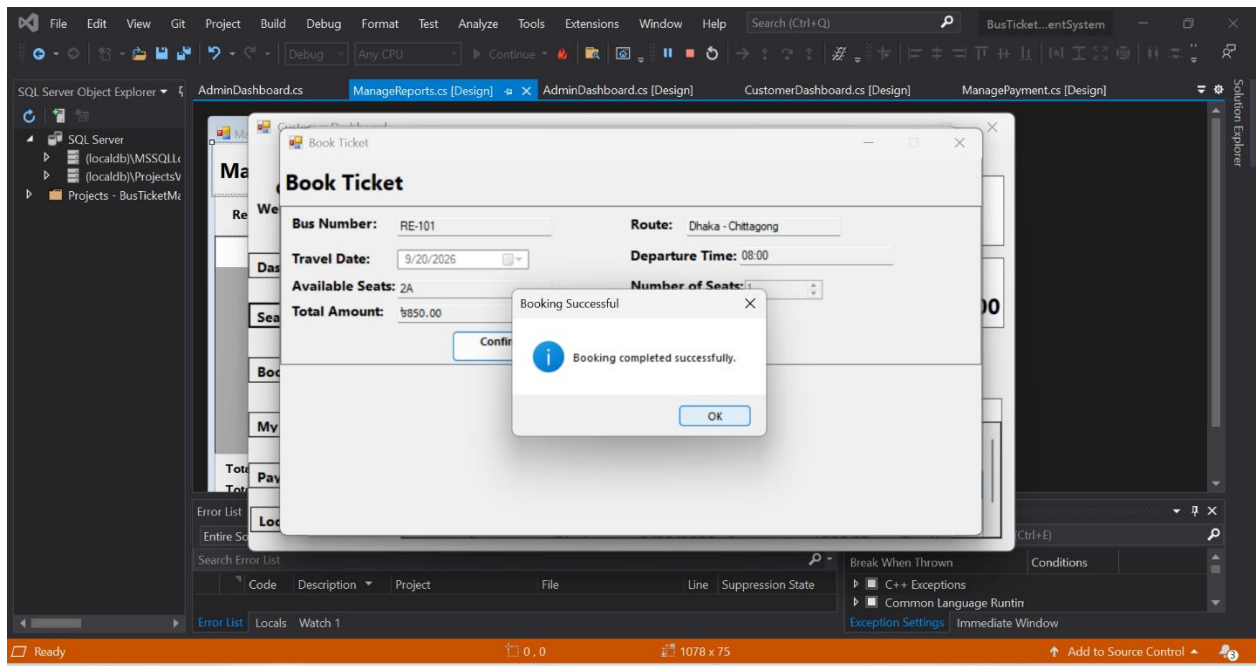
These given screenshots are for Customer given one by one -----











Chapter 7 — Work Distribution

The project work was distributed between the two group members according to their respective responsibilities in system development, interface design, database implementation, testing, and documentation.

Member Name	ID	Contribution
KAZI FAHMID	24-58484-2	Responsible for the overall system development and implementation. Contributed to system planning, SQL Server database design and implementation, database relationships, SQL queries, role-based authentication, Super Admin module, Operator/Admin module, Customer module, bus management, route management, schedule management, seat selection, booking workflow, payment processing, operator commission and payment reporting, system integration, debugging, and functional testing.

Member Name	ID	Contribution
Jannatul Mawa Rodela	24-58743-2	Contributed to user interface design and refinement, including role selection, login/registration, dashboard, customer, and operator interfaces. Also contributed to usability review, testing support, identification of interface and functional issues, project documentation, preparation of screenshots and diagrams, and final presentation preparation.

Chapter 8 — Conclusion & Future Work

8.1 Conclusion

The Bus Ticket Management System was developed as a C# Windows Forms desktop application with SQL Server LocalDB as the database platform. The system provides separate functionalities for Super Admins, Operators/Admins, and Customers.

The completed system supports major bus ticket management activities, including user authentication, operator management, bus management, route management, schedule management, seat selection, ticket booking, payment processing, operator commission management, and reporting. The role-based interface allows each type of user to access the functions relevant to their responsibilities.

The project also demonstrates the practical use of relational database design, SQL queries, foreign key relationships, Windows Forms interfaces, transaction-based booking and payment operations, and integration between a desktop application and a SQL Server database.

Overall, the developed system provides a functional foundation for managing bus services and online-style ticket booking within a desktop environment.

8.2 Future Work

Although the current system provides the core functionality required for the project, several features can be implemented in future versions to make the system more complete and suitable for real-world use.

1. Online Payment Gateway

A real payment gateway such as mobile financial services, cards, or online banking could be integrated instead of using a simulated/local payment process. This would allow customers to make real-time payments securely.

2. Password Security

The current authentication system can be improved by implementing secure password hashing and salting instead of storing passwords in directly comparable form. Additional security measures such as account lockout and stronger password policies could also be introduced.

3. Online/Web-Based Version

The desktop application could be extended into a web-based system so that customers can search for buses and purchase tickets remotely using a browser without installing the desktop application.

4. Mobile Application

A mobile application could be developed for Android and iOS platforms. Customers would then be able to search schedules, select seats, book tickets, make payments, and receive notifications from their mobile devices.

5. Email and SMS Notifications

The system could send automated notifications for successful bookings, payment confirmations, schedule changes, cancellations, and upcoming journeys through email or SMS.

6. Advanced Reporting

Future versions could include more advanced reports and dashboards, such as daily and monthly sales analysis, operator performance, popular routes, occupancy rates, revenue trends, and cancellation statistics.

7. Improved Seat Layout Management

A more advanced seat-layout designer could be implemented so that operators can configure different bus layouts visually for AC, Non-AC, and Sleeper buses. This would allow the seat arrangement to match the actual physical layout of each bus.

8. Cloud Database and Multi-User Access

The current LocalDB-based system could be migrated to a centralized cloud or network database. This would allow multiple operators and administrators to access the same system simultaneously from different locations.

8.3 Expected Challenges

Several challenges may arise during future development and expansion of the system.

Security Challenges

Integrating real payments and storing customer information will require stronger authentication, authorization, encryption, and protection against unauthorized access.

Database Scalability

As the number of customers, bookings, buses, and transactions increases, database performance and query optimization will become increasingly important.

Real-Time Seat Availability

Supporting multiple users simultaneously will require proper transaction handling and concurrency control to prevent two customers from booking the same seat at the same time.

Payment Integration

Connecting external payment gateways may require API integration, transaction verification, error handling, refund processing, and compliance with the payment provider's requirements.

System Availability

A web or cloud-based version would require reliable hosting, backup mechanisms, monitoring, and recovery procedures to ensure that the service remains available.

Different Bus Configurations

Supporting multiple physical bus layouts can be challenging because different operators may use different arrangements for regular, AC, and sleeper buses. A flexible seat-layout system would therefore be required.

Data Privacy

As the system expands, protecting customer information, payment-related information, and account credentials will become an important requirement.

8.4 Final Statement

The current Bus Ticket Management System successfully demonstrates the core concepts required to develop a database-driven ticket management application. Future improvements can transform the system from a desktop-based academic project into a more scalable, secure, and real-world transportation platform.

